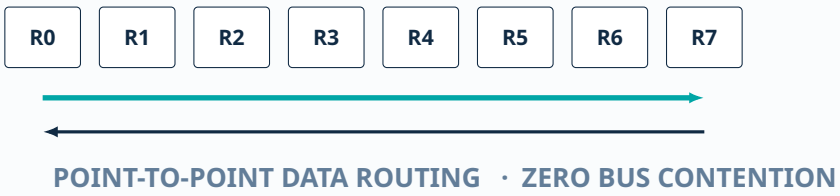
HIGH-PERFORMANCE

16-BIT ARCHITECTURE

LIM M2

Архитектурное руководство по процессору

Высокодетерминированная 16-разрядная процессорная архитектура с чистым мультиплексорным трактом данных и расширенной 24-битной адресацией.



OFFICIAL REFERENCE SPECIFICATION

Редакция 0.2 — Октябрь 2026 г.

Спецификация ядра LIM M2. Все права защищены.

Contents

1 Статус документа и архитектурные цели **4**
1.1 Условные обозначения и сокращения 4
1.2 Единая философия формата инструкций 4

2 Архитектурный обзор процессора LIM M2 **5**
2.1 Концептуальная схема тракта данных 6

3 Программно видимые регистры и состояние процессора **7**
3.1 Топология регистра флагов и управления (FREG) 7

4 Формат машинных инструкций **8**
4.1 Универсальный битовый шаблон машинной команды 8
4.2 Класс I: Однобайтовые инструкции (1 байт, 0 дополнительных октетов) 9
4.3 Класс II: Трёхоперандные арифметические инструкции (2 байта) 9
4.4 Класс III: Двухоперандные логические инструкции (2 байта) 9
4.5 Класс IV: Межрегистровое перемещение LRR (2 байта) 10
4.6 Класс V: Аппаратный компаратор CMP (2 байта) 10
4.7 Класс VI: Аппаратный циклический сдвиг на D-триггерах (BSL, BSR) 10
4.8 Класс VII: Операции со стеком (PUSH, POP) 10
4.9 Класс VIII: Манипуляция специальными регистрами (MSP, Опкод 26) 11
4.9.1 Формат 8.1: Загрузка во флаговый регистр FREG (MOD = 0, REG = 00) . . . 11
4.9.2 Формат 8.2: Побитовая логика FREG (MOD = 1, REG = 00) 11
4.9.3 Формат 8.3: Загрузка указателей PTREG и SREG (MOD = 0, REG = 01 / 10) . 11
4.9.4 Формат 8.4: Арифметика смещения PTREG и SREG (MOD = 1, REG = 01 / 10) 12
4.9.5 Формат 8.5: Загрузка 32-битных векторных регистров (MOD = 0, REG = 11) 12
4.10 Класс IX: Инструкции ветвления и переходов (2–3 байта) 12
4.11 Класс X: Инструкции обмена с памятью LMR и LRM (2–4 байта) 13

5 Сводная таблица команд ядра LIM M2 **13**

6 Детальная спецификация системы команд **14**
NOP — Аппаратная пустая операция и сброс арифметической цепочки 15
ADD — Сложение с автоматическим каскадированием переноса 17
SUB — Вычитание с автоматическим каскадированием заёма 19
MUL — Целочисленное умножение с сохранением старшей части в регистре АЛУ . 21
DIV — Целочисленное деление с сохранением остатка во внутреннем регистре АЛУ 23
LMR — Загрузка данных из памяти и внешних устройств в регистры 25
LRM — Запись данных из регистров в память и внешние устройства 27
LRR — Межрегистровое перемещение (копирование из точки Б в точку А) 29

CMP — Аппаратное сравнение операндов . . . 31
JMP — Безусловный переход по адресу . . . 33
JCR — Условный переход по флагу переноса . . . 35
JNZ — Условный переход при неравенстве нулю . . . 37
JZ — Условный переход при равенстве нулю . . . 39
AND — Побитовое логическое И . . . 41
OR — Побитовое логическое ИЛИ . . . 43
NAND — Побитовое логическое И-НЕ . . . 45
NOR — Побитовое логическое ИЛИ-НЕ . . . 47
NOT — Побитовое инвертирование (дополнение до 1) . . . 49
XOR — Побитое исключающее ИЛИ . . . 51
XNOR — Побитовая эквивалентность . . . 53
INC — Инкремент операнда на единицу . . . 55
DEC — Декремент операнда на единицу . . . 57
PUSH — Сохранение слова в стек . . . 59
POP — Извлечение слова из вершины стека . . . 61
BSL — Аппаратный сдвиг влево (итеративный регистр) . . . 63
BSR — Аппаратный сдвиг вправо (итеративный регистр) . . . 65
MSP — Манипуляция спецрегистрами (FREG, PTREG, SREG, VEC) . . . 67
MOD — Целочисленный остаток от деления (modulo) . . . 69
INTR — Программное прерывание по вектору из стека векторов ядра . . . 71
CALL — Вызов подпрограммы с сохранением PC в стек . . . 73
RET — Возврат из подпрограммы с извлечением PC из стека . . . 75
HALT — Аппаратный останов процессора . . . 77

7 Физический интерфейс и цоколёвка (50-выводный корпус) 79
7.1 Функциональное назначение внешних линий . . . 79

8 Подсистема прерываний и векторная обработка 80
8.1 Единое пространство 16 векторов прерываний . . . 80
8.2 Аппаратный протокол квитирования (RDI → IA → D0–D7) . . . 80
8.3 Сохранение контекста в 24-битном стеке и вход в обработчик . . . 81
8.4 Возврат из прерывания . . . 81

9 Язык ассемблера и компилятор LIM M2 (`limasm.py`) 82
9.1 Именование регистров и суффиксы размера операндов . . . 82
9.2 Арифметические и унарные регистровые инструкции . . . 82
9.3 Загрузка и выгрузка памяти (LMR и LRM): числа, ссылки `[...]` и пары регистров 82
9.4 Манипуляция специальными регистрами (MSP) и многократное разыменование указателей . . . 83
9.5 Использование CLI-компилятора (`.bin` и `.hex`) . . . 84

10 Заключение 84

1 Статус документа и архитектурные цели

Настоящий документ является официальным техническим описанием и архитектурным стандартом микропроцессора **LIM M2**. Архитектура спроектирована с прицелом на бескомпромиссную надежность, математическую строгость и высокую тактовую частоту.

В основу тракта данных LIM M2 заложен фундаментальный принцип **полного отказа от внутренних шин третьего состояния (Tri-State / Hi-Z)**. Внутренние потоки данных построены исключительно на детерминированных двухтактных мультиплексорных деревьях (Point-to-Point MUX Routing), что гарантирует полное отсутствие сквозных токов, шинных конфликтов (bus contention) и паразитных емкостных задержек.

> **Архитектурная нумерация команд**
> Система команд LIM M2 использует непрерывную 5-битную сетку первичных кодов: `NOP`=0, `ADD`=1, ..., `MSP`=26. Коды 27–31 зарезервированы для аппаратных расширений следующего поколения.

1.1 Условные обозначения и сокращения

Обозначение	Архитектурный смысл
`dst`	Целевой регистр-приёмник результата операции
`src, srcA, srcB`	Регистры-источники входных операндов
`imm`	Непосредственное скалярное значение, встроенное в тело инструкции
`addr`	Эффективный адрес памяти (16-битный базовый или 24-битный расширенный)
`cfg`	3-битное поле функционального модификатора операции
`ZF, CF, OF`	Аппаратные флаги нуля, переноса и знакового переполнения регистра FREG

1.2 Единая философия формата инструкций

Архитектура процессора LIM M2 строго придерживается универсальной парадигмы: **сначала всегда указывается целевой приёмник (куда), а затем входные операнды (откуда)**. Это обеспечивает строгий, логичный и интуитивно понятный синтаксис:

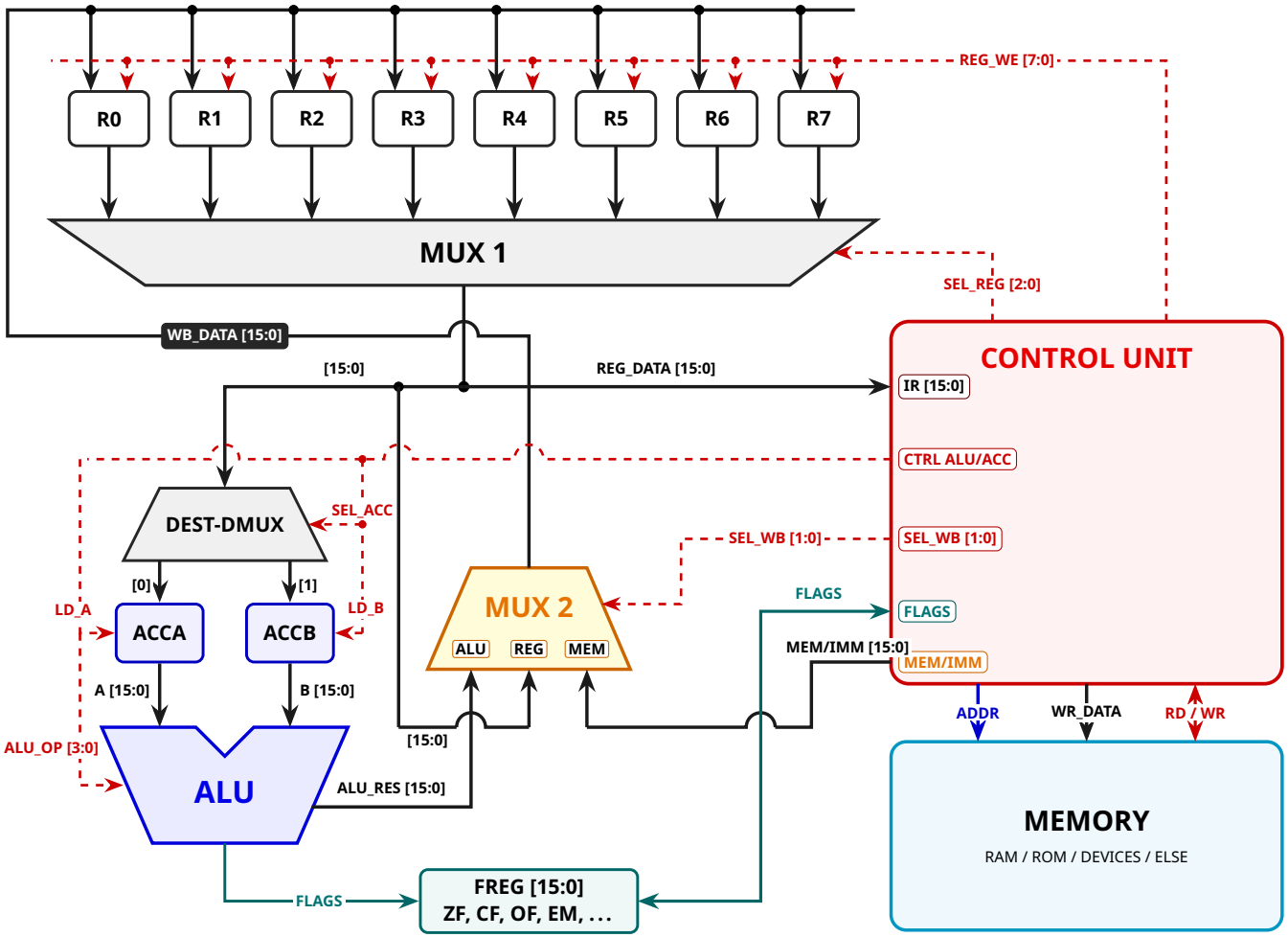
- **Арифметические операции (3 операнда):** `ADD Rd, Rs1, Rs2` означает Rd ← Rs1 + Rs2 (например, `ADD R0, R1, R2` вычисляет $R0 = R1 + R2$); аналогично для `SUB`, `MUL`, `DIV`.
- **Логические операции (2 операнда):** `AND Rd, Rs` означает Rd ← Rd AND Rs (например, `AND R0, R1` вычисляет $R0 = R0 \,\&\, R1$); аналогично для `OR`, `NAND`, `NOR`, `XOR`, `XNOR`.
- **Однобайтовые унарные операции (1 операнд):** `INC Rd` (Rd ← Rd+1), `DEC Rd` (Rd ← Rd−1) и `NOT Rd` (Rd ←∼ Rd). Имеют ровно 1 аргумент — целевой регистр Rd, модифицируемый на месте. Кодируются ровно в 1 байте (`[7:3]` опкод, `[2:0]` регистр Rd), второй байт из шины памяти не запрашивается.
- **Межрегистровые пересылки:** `LRR Rd, Rs` означает прямое копирование из точки Б (источник Rs) в точку А (приёмник Rd): Rd ← Rs (например, `LRR R0, R1` означает $R0 = R1$).
- **Загрузка из памяти (LMR):** первый аргумент Rd задаёт целевой регистр процессора, куда сохраняются считанные данные.

2 Архитектурный обзор процессора LIM M2

LIM M2 представляет собой высокоэффективный 16-разрядный микропрограммный процессор общего назначения с развитой целочисленной арифметикой. Ключевые архитектурные достоинства процессора включают:

- **Чистая мультиплексорная маршрутизация данных:** Внутренний тракт полностью избавлен от третьего состояния. Передача данных осуществляется активными драйверами через каскад селекторов **MUX 1**, **DEST-DMUX** и **MUX 2**, что делает архитектуру идеально совместимой с современными полупроводниковыми техпроцессами (ASIC) и быстрыми ПЛИС (FPGA).
- **Двухаккумуляторный исполнительный узел (Dual Accumulator Staging):** Регистры ACCA и ACCB обеспечивают надёжную фиксацию операндов перед АЛУ, изолируя переходные процессы вычислений от основного регистрового файла.
- **Аппаратное каскадирование длинной арифметики (ALU Auto-Chaining):** Внутренний тракт АЛУ оснащён остаточным аккумулятором `ALU_RESIDUAL`. При возникновении переноса ($+1$ при сложении $> 2^{16} - 1$), заёма (-1 при разности < 0) или избыточных старших разрядов умножения/деления, остаток фиксируется аппаратно и автоматически суммируется или вычитается при следующей математической операции. Это обеспечивает непрерывную многобайтовую арифметику (32, 48, 64+ бит) без отдельных команд ADC/SBB. Сброс цепочки выполняется командой `NOP`.
- **Прямой тракт пересылки (Reg-to-Reg Bypass):** Обходной канал от MUX 1 напрямую в MUX 2 позволяет осуществлять копирование данных между регистрами без задействования и блокировки вычислительных ресурсов АЛУ.
- **Нативная 32-битная адресация памяти (до 4 ГБ):** Нативная 32-разрядная магистраль адреса (`A0`–`A31`) и специализированные 32-битные регистры-указатели `PTREG` (указатель данных с автоинкрементом) и `SREG` (указатель стека), а также 32 системных векторных регистра (`$ir0..$ir15`, `$pv0..$pv15`) обеспечивают процессору прямое линейное обращение ко всей физической памяти объёмом 4 Гбайта без необходимости мультиплексирования шин, переключения страниц или банков.
- **Высокочастотный потенциал:** Отсутствие емкостной нагрузки от распределённых шин позволяет ядру достигать базовой рабочей частоты **200 МГц**, а при трансляции в специализированный кремний (ASIC) ориентироваться на частоты **700 МГц и выше**.

2.1 Концептуальная схема тракта данных



3 Программно видимые регистры и состояние процессора

Архитектурное состояние процессора LIM M2 представлено набором ортогональных регистров, сгруппированных по функциональному назначению:

Регистр	Разрядность	Архитектурная роль и особенности
R0–R7	16 бит	Регистры общего назначения (GPR). Селектор MUX 1 обеспечивает сверхбыстрое бесконфликтное считывание; фиксация результата происходит по стробу с шины WB_DATA.
SREG	24 бита	Системный регистр стека (Stack Register). Поддерживает эффективное аппаратное отслеживание глубины стековых кадров с прямой 24-битной адресацией стекового пространства (до 16 Мбайт).
PTREG	24 бита	Высокоточный регистр-указатель (Pointer Register). Обеспечивает прямое линейное обращение к памяти объёмом до 16 Мбайт с аппаратным автоинкрементом по флагу AIP.
FREG	16 бит	Регистр состояния и управления (Processor Status & Control Register). Объединяет флаги арифметических результатов и биты конфигурации режимов ядра.

3.1 Топология регистра флагов и управления (FREG)

Регистр FREG является ключевым элементом контроля математического тракта и системных режимов:

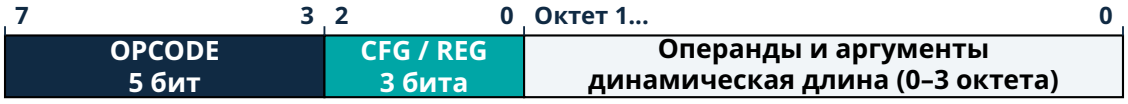
Бит	Мнемоника	Функциональное описание
0	ZF	Zero Flag: истинен при нулевом результате операции.
1	CF	Carry Flag: аппаратный флаг переноса / заёма в арифметике.
2	OF	Overflow Flag: индикатор знакового переполнения дополнительного кода.
3	EM	Error Math: аппаратная ловушка математических исключений (деление на ноль).
4–5	CMP	Compare Result: результат аппаратного компаратора (00 — сброс/нет сравнения/ошибка, 01 — меньше, 10 — равно, 11 — больше).
6–7	CLK	Clock Mode: выбор профиля делителя тактовой частоты ядра.
8	PIE	Program Interrupt Enable: глобальное разрешение программных прерываний.
9	MIE	Master Interrupt Enable: разрешение маскируемых внешних прерываний.
10	DWR	Disable Wait for Ready: форсированный безакцептный обмен без ожидания шины.
11	PRV	Privilege Mode: 0 = User Mode, 1 = Supervisor Mode (аппаратный старт в 1; защита через 8 Зон Привилегий $pv0..$pv15).
12	AIP	Auto-Increment PTREG: разрешение аппаратного инкремента указателя PTREG при обращении.
13	DBIT	Displaced Bit: буфер вытесненного бита сдвигового узла.
14	SMOD	Shift Mode: управление режимом сдвига (0 — заполнение нулями, 1 — циклический проход через DBIT).
15	INTR	In Interrupt: флаг нахождения процессора в прерывании (при 1 новые прерывания аппаратно заблокированы).

4 Формат машинных инструкций

Инструкции микропроцессора LIM M2 обладают ортогональным, модульным форматом с детерминированным декодированием. Базовый опкод всегда занимает старшие 5 бит первого октета, что гарантирует мгновенную аппаратную диспетчеризацию в микропрограммном блоке управления.

4.1 Универсальный битовый шаблон машинной команды

Каждая машинная инструкция начинается с базового заголовка фиксированного размера, за которым следуют динамические операнды и расширения:



Младшие 3 бита первого октета ([2:0]) имеют двойственное детерминированное назначение:

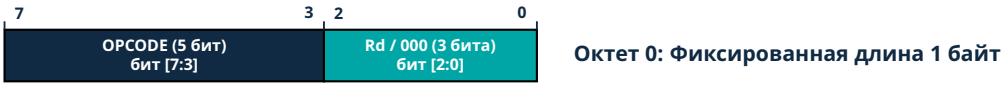
- В операциях с регистрами (арифметика, унарные команды, логика, сдвиги, пересылка, стек) биты $[2:0]$ напрямую кодируют **первый регистровый аргумент** (Rd или Rs1), что позволяет оперировать полным набором из 8 регистров R0–R7 без дополнительных накладных расходов.
- В командах, требующих выбора аппаратного режима или селектора памяти/спецрегистров (LMR, LRM, MSP, JMP, JCR, JNZ, JZ), биты $[2:0]$ выступают конфигуратором режима (CFG / MOD / SRC / DST).

- В безаргументных управляющих инструкциях (`NOP`, `HALT`, `CLI`, `STI`, `RET`) эти биты содержат `000`.

4.2 Класс I: Однобайтовые инструкции (1 байт, 0 дополнительных октетов)

Инструкции управления (`NOP`, `HALT`, `CLI`, `STI`, `RET`) и унарные регистровые операции (`NOT`, `INC`, `DEC`) считываются за ровно один такт выборки. Второй байт из шины памяти не запрашивается:

- Для безаргументных команд младшие 3 бита содержат `000`.
- Для унарных команд (`INC Rd`, `DEC Rd`, `NOT Rd`) младшие 3 бита (`[2:0]`) содержат индекс целевого регистра Rd (`R0`–`R7`, единственный аргумент).



4.3 Класс II: Трёхоперандные арифметические инструкции (2 байта)

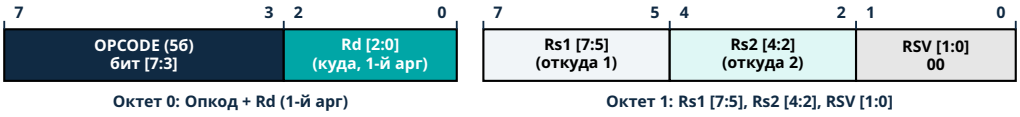
Команды вычислений (`ADD`, `SUB`, `MUL`, `DIV`, `MOD`) строго реализуют архитектурную парадигму **«сначала приёмник, затем операнды»**:

$$\texttt{ADD Rd, Rs1, Rs2} \quad \Longrightarrow \quad \texttt{Rd} \leftarrow \texttt{Rs1} + \texttt{Rs2} + \texttt{ALU_RESIDUAL}$$

$$\texttt{MOD Rd, Rs1, Rs2} \quad \Longrightarrow \quad \texttt{Rd} \leftarrow \texttt{Rs1} \pmod{\texttt{Rs2}}$$

Первый аргумент команды (целевой регистр назначения Rd) размещается непосредственно в младших 3 битах первого октета, что позволяет адресовать полные 8 регистров `R0`–`R7` для всех трёх операндов:

- **Октет 0:** биты $[7:3]$ — код операции (5 бит); биты $[2:0]$ — целевой регистр назначения Rd (3 бита, `R0`–`R7`, первый аргумент).
- **Октет 1:** биты $[7:5]$ — первый операнд-источник `Rs1` (3 бита, `R0`–`R7`); биты $[4:2]$ — второй операнд-источник `Rs2` (3 бита, `R0`–`R7`); биты $[1:0]$ — резерв (`00`).

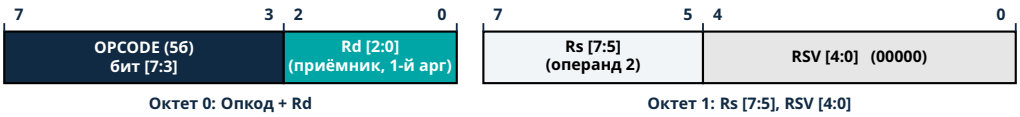


4.4 Класс III: Двухоперандные логические инструкции (2 байта)

Логические операции (`AND`, `OR`, `NAND`, `NOR`, `XOR`, `XNOR`) имеют ровно два операнда:

$$\texttt{AND Rd, Rs} \quad \Longrightarrow \quad \texttt{Rd} \leftarrow \texttt{Rd AND Rs}$$

- **Октет 0:** биты $[7:3]$ — код операции; биты $[2:0]$ — целевой регистр-приёмник Rd (`R0`–`R7`, первый аргумент).
- **Октет 1:** биты $[7:5]$ — второй регистр-операнд `Rs` (`R0`–`R7`); биты $[4:0]$ — зарезервированы (00000_2).



4.5 Класс IV: Межрегистровое перемещение LRR (2 байта)

Инструкция `LRR Rd, Rs` копирует данные из источника (точка Б, Rs) в приёмник (точка А, Rd):

$$\texttt{LRR Rd, Rs} \quad \Longrightarrow \quad \texttt{Rd} \leftarrow \texttt{Rs}$$

Перемещение осуществляется через прямой аппаратный обходной тракт **MUX 1 → MUX 2** минуя АЛУ за один микротакт, без изменения регистра флагов FREG. Формат октетов: Октет 0 содержит опкод `00111` и целевой регистр Rd (`[2:0]`), Октет 1 содержит регистр-источник Rs (`[7:5]`) и резерв (`[4:0]` = `00000`).

4.6 Класс V: Аппаратный компаратор CMP (2 байта)

Инструкция `CMP Rs1, Rs2` сравнивает значения регистров без разрушения содержимого:

- **Октет 0:** код операции `01000`, биты $[2:0]$ задают первый операнд-источник `Rs1` (первый аргумент).
- **Октет 1:** биты $[7:5]$ задают второй операнд-источник `Rs2`; биты $[4:0]$ содержат `00000`.

Результат сравнения фиксируется в специализированном 2-битном аппаратном поле регистра состояния `FREG.CMP` (биты $[5:4]$):

Значение FREG.CMP	Условие	Архитектурный смысл
`01` (01_2)	Rs1 $<$ Rs2	Первый операнд строго меньше второго
`10` (10_2)	Rs1 $=$ Rs2	Значения операндов равны
`11` (11_2)	Rs1 $>$ Rs2	Первый операнд строго больше второго
`00` (00_2)	Исходное / Сброс	Ничего не сравнивалось / ошибка компаратора

4.7 Класс VI: Аппаратный циклический сдвиг на D-триггерах (BSL, BSR)

Команды сдвига `BSL Rd, shift` и `BSR Rd, shift` реализованы в кремнии как **итеративный 16-разрядный регистр сдвига на D-триггерах** (≈ 300 транзисторов вместо $\approx 30\,000$ у матричного баррельного узла):

- Сдвиг выполняется аппаратно через такт-за-тактом (`for` в железе): сдвиг на 16 бит требует ровно на 15 тактов больше, чем сдвиг на 1 бит.
- Октет 0: старшие 5 бит задают опкод (BSL: `11000`, BSR: `11001`), а младшие 3 бита (`[2:0]`) прямо указывают сдвигаемый регистр Rd.
- Октет 1: весь байт (`[7:0]`) указывает число тактов сдвига shift (0–16).
- Поведение битов определяется регистром FREG: при `SMOD` $= 1$ включена аппаратная ротация через вытесненный бит; при `SMOD` $= 0$ пустые разряды заполняются битом из `FREG.DBIT`.



4.8 Класс VII: Операции со стеком (PUSH, POP)

Команды стека сохраняют регистр в память по 24-битному указателю SREG:

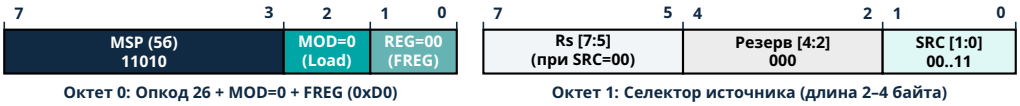
- **Октет 0:** код операции (PUSH: `10110`, POP: `10111`), биты $[2:0]$ прямо задают регистр операнда (Rs при сохранении, Rd при извлечении, первый аргумент).
- **Октет 1:** биты $[7:0]$ содержат 00000000_2 (резерв).

4.9 Класс VIII: Манипуляция специальными регистрами (MSP, Опкод 26)

Инструкция MSP (Опкод 11010, 26_{10}) обеспечивает универсальный модульный доступ ко всем системным, адресным и защитным ресурсам ядра: FREG, PTREG, SREG, а также 32 системным векторам прерываний и границ Зон Привилегий (`$ir0..$ir15`, `$pv0..$pv15`). Длина команды составляет от 2 до 6 байтов в зависимости от выбранного режима.

4.9.1 Формат 8.1: Загрузка во флаговый регистр FREG (MOD = 0, REG = 00)

Команда `MSP.ld $freg, <src>` выполняет прямую инициализацию флагов.

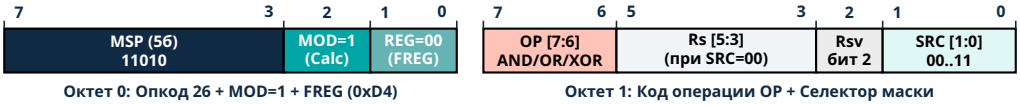


Октет 0: Опкод 26 + MOD=0 + FREG (0xD0) Октет 1: Селектор источника (длина 2–4 байта)

- SRC $= 00_2$ (2 байта): загрузка из регистра РОН Rs (`MSP.ld $freg, $rs`).
- SRC $= 01_2$ (4 байта): непосредственная константа imm16 в октетах 2–3 (`MSP.ld $freg, 0x0000`).
- SRC $= 10_2$ (2 байта): чтение из памяти по текущему указателю `[$ptreg]` (`MSP.ld $freg, [$ptreg]`).
- SRC $= 11_2$ (4 байта): чтение из памяти по прямому 16-битному адресу `[addr16]` (`MSP.ld $freg, [0x1000]`).

4.9.2 Формат 8.2: Побитовая логика FREG (MOD = 1, REG = 00)

Команды `MSP.and`, `MSP.or`, `MSP.xor`, `MSP.xnor` применяют побитовую маску к флагам.

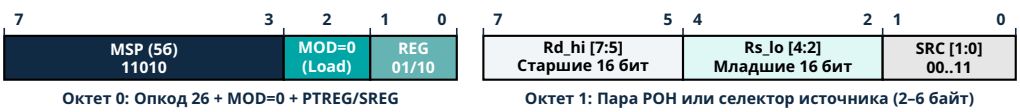


Октет 0: Опкод 26 + MOD=1 + FREG (0xD4) Октет 1: Код операции OP + Селектор маски

- **Код операции OP**$[7:6]$**:** 00_2 = AND, 01_2 = OR, 10_2 = XOR, 11_2 = XNOR.
- **Аппаратная защита режима супервизора (PRV):** Процессор аппаратно стартует при включении питания и сбросе в режиме супервизора (PRV=1). Начальный код (загрузчик/ядро) настраивает системные векторы (`$ir0..$ir15`) и границы Зон Привилегий (`$pv0..$pv15`), после чего сбрасывает PRV=0 и передает управление пользовательскому коду. В пользовательском режиме (PRV=0) включение PRV=1 и модификация регистров зон аппаратно разрешены исключительно если текущий PC находится строго внутри одной из 8 зарегистрированных Зон Привилегий ($\exists k \in [0..7] : \$pv_{2k} \leq \text{PC} \leq \$pv_{2k+1}$). **Важное архитектурное правило:** при выходе за пределы Зоны Привилегий режим супервизора PRV=1 автоматически **не сбрасывается** — ОС/программа обязана явно сбросить его перед возвратом пользователю.

4.9.3 Формат 8.3: Загрузка указателей PTREG и SREG (MOD = 0, REG = 01 / 10)

Команды `MSP.ld $ptreg` (REG $= 01_2$, опкод 0xD1) и `MSP.ld $sreg` (REG $= 10_2$, опкод 0xD2) выполняют прямую инициализацию 32-битных линейных указателей.

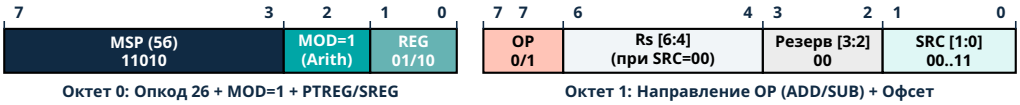


Октет 0: Опкод 26 + MOD=0 + PTREG/SREG Октет 1: Пара РОН или селектор источника (2–6 байт)

- SRC $= 00_2$ (2 байта): сборка адреса из пары РОН Rd_hi:Rs_lo (`MSP.ld $ptreg, [$0:$1]`).
- SRC $= 01_2$ (4–6 байт): непосредственный адрес `imm32` (6 байт) или `imm16` (4 байта).
- SRC $= 10_2$ (**2 байта**): **нативное 1-тактовое разыменование указателя** `[$ptreg]` (`PTREG <- MEM[PTREG]`).
- SRC $= 11_2$ (4–6 байт): загрузка указателя из памяти по абсолютному адресу `[addr]` (`MSP.ld $ptreg, [0xA000]`).

4.9.4 Формат 8.4: Арифметика смещения PTREG и SREG (MOD = 1, REG = 01 / 10)

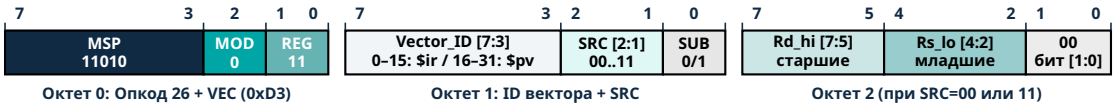
Команды `MSP.add` (OP $= 0$) и `MSP.sub` (OP $= 1$) осуществляют быстрое аппаратное позиционирование указателей буфера и вершины стека.



- OP $= 0$: ADD (+смещение к указателю), OP $= 1$: SUB (-смещение от указателя).
- Источник смещения: регистр РОН SRC $= 00_2$ (2 байта), непосредственная константа SRC $= 01_2$ (4 байта), чтение смещения по SRC $= 10_2$ (`[$ptreg]`, 2 байта) или SRC $= 11_2$ (`[addr16]`, 4 байта).

4.9.5 Формат 8.5: Загрузка 32-битных векторных регистров (MOD = 0, REG = 11)

Команда `MSP.ld` загружает 32-битные адреса в один из 32 векторных регистров: `$ir0`..`$ir15` (векторы прерываний, ID 0–15) и `$pv0`..`$pv15` (границы Зон Привилегий, ID 16–31).



- **Вариант 5.1 (SRC $= 00_2$, SUB $= 0$, 3 байта):** загрузка вектора из пары РОН Rd_hi:Rs_lo (`MSP.ld $ir0, [$0:$1]`).
- **Вариант 5.2 (SRC $= 01_2$, SUB $= 0$, 6 байтов):** прямой 32-битный вектор адреса `imm32` в октетах 2–5 (`MSP.ld $pv0, 0x00008000`).
- **Вариант 5.3 (SRC $= 10_2$, SUB $= 0$, 2 байта):** чтение вектора из памяти по указателю `[$ptreg]` (`MSP.ld $ir1, [$ptreg]`).
- **Вариант 5.4 (SRC $= 11_2$, SUB $= 0$, 3 байта):** чтение вектора из памяти по 32-битному указателю в паре РОН `[$rd:$rs]` (`MSP.ld $ir2, [$0:$1]`).
- **Вариант 5.5 (SRC $= 11_2$, SUB $= 1$, 6 байтов):** чтение вектора из памяти по прямому 32-битному адресу `[addr32]` (`MSP.ld $ir0, [0x00005000]`).

4.10 Класс IX: Инструкции ветвления и переходов (2–3 байта)

Команды переходов (JMP, JCR, JNZ, JZ) поддерживают три взаимоисключающих режима адресации, задаваемых полем CFG (биты $[2:0]$ октета 0):

- **MOD = 0 (000): Непосредственный 16-битный адрес (длина 3 байта).** За октетом 0 следуют октеты 1 и 2, формирующие абсолютный 16-разрядный адрес целевой точки перехода в текущем 64-КБ сегменте.
- **MOD = 1 (001): Косвенный регистровый переход (длина 2 байта).** Октет 1 указывает целевой регистр Rs (биты $[7:5]$), значение которого загружается в PC.
- **MOD = 2 (010): Расширенный 24-битный переход по указателю PTREG.** Переход производится по физическому 24-битному адресу из системного регистра PTREG.

4.11 Класс X: Инструкции обмена с памятью LMR и LRM (2–4 байта)

Команды загрузки (LMR) и сохранения (LRM) обладают наивысшей степенью адаптивности. Поле конфигурации октета 0 (`[2:0]`) кодирует:

- Бит $[2]$: размер операнда SZ ($0 = 8$ бит / байт, $1 = 16$ бит / слово).
- Биты $[1:0]$: источник/приёмник SRC / DST:
  - **0: Непосредственные данные (Inline Data):** 2 байта при SZ $= 0$ (байт за командой) или 3 байта при SZ $= 1$ (16-битное слово за командой).
  - **1: Непосредственный указатель (Imm Pointer):** 4 байта. Октет 1 задаёт рабочий регистр, а октеты 2–3 хранят 16-битный абсолютный адрес данных.
  - **2: Системный 24-битный указатель PTREG:** 2 байта. Октет 1 задаёт целевой регистр. При активном флаге `FREG.AIP` указатель PTREG аппаратно инкрементируется за 0 дополнительных тактов.
  - **3: Составной 24-битный адрес из пары регистров:** 2 байта. Октет 1 содержит биты $[7:5]$ (`Arg1` / старшие биты адреса) и биты $[4:2]$ (`Arg2` / младшие 16 бит адреса). В команде LMR регистр `Arg1` после чтения перезаписывается считанными данными.

5 Сводная таблица команд ядра LIM M2

№	Код	Мнемоника	Класс	Назначение операции
0	00000	NOP	Управление	Пустая операция и сброс остаточного регистра АЛУ
1	00001	ADD	Арифметика	Сложение с автокаскадированием переноса через остаток АЛУ
2	00010	SUB	Арифметика	Вычитание с автокаскадированием заёма через остаток АЛУ
3	00011	MUL	Арифметика	Умножение с фиксацией старших разрядов в остатке АЛУ
4	00100	DIV	Арифметика	Деление с фиксацией остатка во внутреннем регистре АЛУ
5	00101	LMR	Память	Загрузка из памяти/ввода в регистр (конфиг. источника 0–3, размер 8/16 бит)
6	00110	LRM	Память	Запись данных из регистра в память/вывод (конфиг. назначения 0–3, размер 8/16 бит)
7	00111	LRR	Пересылка	Прямое межрегистровое копирование через обходной MUX
8	01000	CMP	Сравнение	Аппаратное сравнение операндов без разрушения данных
9	01001	JMP	Переход	Безусловный переход (прямой, косвенный, 24-битный)
10	01010	JCR	Переход	Условный переход по состоянию флага переноса CF
11	01011	JNZ	Переход	Условный переход при ненулевом результате (ZF = 0)
12	01100	JZ	Переход	Условный переход при нулевом результате (ZF = 1)
13	01101	AND	Логика	Побитовое логическое умножение (И)
14	01110	OR	Логика	Побитовое логическое сложение (ИЛИ)
15	01111	NAND	Логика	Побитовая инверсия логического И
16	10000	NOR	Логика	Побитовая инверсия логического ИЛИ
17	10001	NOT	Логика	Побитовое инвертирование операнда (дополнение до 1)
18	10010	XOR	Логика	Побитовое исключающее ИЛИ
19	10011	XNOR	Логика	Побитовая эквивалентность

№	Код	Мнемоника	Класс	Назначение операции
20	10100	INC	Арифметика	Аппаратный инкремент операнда на единицу
21	10101	DEC	Арифметика	Аппаратный декремент операнда на единицу
22	10110	PUSH	Стек	Размещение регистра в стековом пространстве
23	10111	POP	Стек	Извлечение вершины стека в целевой регистр
24	11000	BSL	Сдвиги	Баррельный сдвиг влево с настраиваемым вводом бит
25	11001	BSR	Сдвиги	Баррельный сдвиг вправо с контролем флага DBIT
26	11010	MSP	Система	Манипуляция специальными регистрами (FREG, PTREG, SREG)
27	11011	MOD	Арифметика	Вычисление остатка от целочисленного деления (modulo)
28	11100	INTR / INT	Система	Программное прерывание (2 байта, младшие 4 бита октета 1 — вектор IR0..IR15)
29	11101	CALL	Переходы	Вызов подпрограммы с сохранением 32-битного PC в стек SREG
30	11110	RET	Переходы	Возврат из подпрограммы с извлечением 32-битного PC из SREG
31	11111	HALT	Управление	Аппаратный останов процессора (HLT)

6 Детальная спецификация системы команд

Ниже представлены исчерпывающие паспорта всех 32 базовых опкодов процессора LIM M2.

NOP

Аппаратная пустая операция и сброс арифметической цепочки

`00000`
Управление

Мнемоника	NOP
Opcode	`00000` (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Управление
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Аппаратная пустая операция и сброс арифметической цепочки.

Архитектурная семантика

```
Сигнатура:  NOP
Операция:  PC <- PC + 1, ALU_RESIDUAL <- 0, FREG.CF <- 0, FREG.OF <- 0
Пример:  NOP ; Сброс цепочки переноса АЛУ
```

Конфигурация и операнды



Октет 0 (1 байт, 0 доп. байт)

Команда фиксированной длины (1 байт). Операнды отсутствуют; поле cfg содержит `000` (зарезервировано для калибровки задержек конвейера). Второй байт из шины не запрашивается.

Влияние на регистр флагов (FREG)

Сбрасывает флаги переноса (CF), переполнения (OF) и заёма каскадной цепочки АЛУ; остальные флаги сохраняются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Служит для тактовой синхронизации конвейера и аппаратного сброса (Flush) внутреннего остаточного регистра АЛУ, разрывая цепочку автопереноса перед новыми независимыми вычислениями.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

ADD

Сложение с автоматическим каскадированием переноса

00001
Арифметика

Мнемоника	ADD
Opcode	00001 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Арифметика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

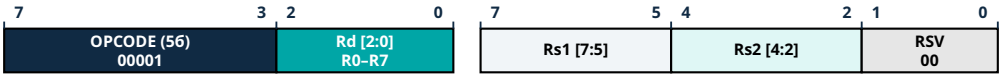
Назначение команды

Сложение с автоматическим каскадированием переноса.

Архитектурная семантика

```
Сигнатура:  ADD arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 + arg2
Операция:  sum <- arg1 + arg2 + ALU_RESIDUAL
arg0 <- sum[15:0], ALU_RESIDUAL <- (sum > 0xFFFF ? 1 :  0)
Пример:  ADD R0, R1, R2 ; R0 = R1 + R2 (регистры R0-R7)
```

Конфигурация и операнды



Октет 0: Опкод + Rd (1-й арг)

Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байта)

Октет 0: код операции 00001, биты $[2:0]$ прямо задают целевой регистр Rd (R0–R7, первый аргумент).
Октет 1: биты $[7:5]$ задают первое слагаемое Rs1 (R0–R7); биты $[4:2]$ задают второе слагаемое Rs2 (R0–R7); биты $[1:0]$ содержат 00 (резерв). Длина команды: ровно 2 байта.

Влияние на регистр флагов (FREG)

Взводит флаг переноса CF и переполнения OF при превышении $2^{16} - 1$; обновляет ZF по значению dst.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.

4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Поддерживает сквозную многобайтовую арифметику (32, 48, 64+ бит). Если результат суммы не помещается в 16 бит ($> 2^{16} - 1$), перенос (+1) фиксируется во внутреннем регистре АЛУ и автоматически суммируется при следующей операции ADD без отдельных инструкций ADC. Младшие 16 бит всегда детерминированно сохраняются в dst. Цепочка сбрасывается инструкцией NOP либо перезаписывается новой математической операцией.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

SUB

Вычитание с автоматическим каскадированием заёма

00010
Арифметика

Мнемоника	SUB
Opcode	`00010` (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Арифметика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Вычитание с автоматическим каскадированием заёма.

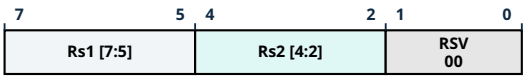
Архитектурная семантика

```
Сигнатура:  SUB arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 - arg2
Операция:  diff <- arg1 - arg2 - ALU_RESIDUAL
arg0 <- diff[15:0], ALU_RESIDUAL <- (diff < 0 ?  1 :  0)
Пример:  SUB R0, R1, R2 ; R0 = R1 - R2 (регистры R0–R7)
```

Конфигурация и операнды



Октет 0: Опкод + Rd (1-й арг)



Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байта)

Октет 0: код операции `00010`, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент, куда сохраняется разность).
Октет 1: биты $[7:5]$ задают уменьшаемое Rs1 (R0–R7); биты $[4:2]$ задают вычитаемое Rs2 (R0–R7); биты $[1:0]$ содержат `00` (резерв). Длина команды: ровно 2 байта.

Влияние на регистр флагов (FREG)

При разности < 0 взводит флаг заёма (Borrow / CF) и OF; обновляет ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Реализует непрерывное многословное вычитание. При возникновении отрицательной разности (< 0) заём (-1) запоминается во внутреннем регистре АЛУ, и в следующей операции SUB АЛУ автоматически вычитает 1 из старшего слова. Результат младшего слова детерминированно фиксируется в dst. Для начала новой независимой разности цепочка прерывается инструкцией NOP.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

MUL

Целочисленное умножение с сохранением старшей части в регистре АЛУ

00011
Арифметика

Мнемоника	MUL
Opcode	00011 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Арифметика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

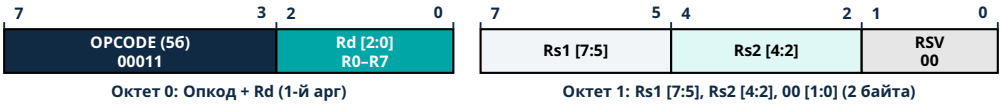
Назначение команды

Целочисленное умножение с сохранением старшей части в регистре АЛУ.

Архитектурная семантика

```
Сигнатура:  MUL arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 * arg2
Операция:  prod <- arg1 * arg2
arg0 <- prod[15:0], ALU_RESIDUAL <- prod[31:16]
Пример:  MUL R0, R1, R2 ; R0 = R1 * R2 (старшие 16 бит в ALU_RESIDUAL)
```

Конфигурация и операнды



Октет 0: Опкод + Rd (1-й арг)

Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байта)

Октет 0: код операции 00011, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент, младшие 16 бит произведения).
Октет 1: биты $[7:5]$ задают первый сомножитель Rs1 (R0–R7); биты $[4:2]$ задают второй сомножитель Rs2 (R0–R7); биты $[1:0]$ содержат 00. Старшие 16 бит произведения автоматически фиксируются в регистре ALU_RESIDUAL. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет ZF; взводит OF/CF, если старшие разряды произведения (prod[31:16]) не равны 0.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.

4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

При умножении младшие 16 бит произведения направляются в целевой регистр dst, а старшие 16 бит сохраняются во внутреннем служебном регистре АЛУ (ALU Residual). При следующей операции сложения сохранённый остаток автоматически суммируется со следующим результатом (аппаратная поддержка MAC / многословных чисел). Старшую часть произведения можно также извлечь в регистр, выполнив сложение регистра с самим собой после очистки. Инструкция NOP очищает остаточный регистр АЛУ.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

DIV

Целочисленное деление с сохранением остатка во внутреннем регистре АЛУ

00100
Арифметика

Мнемоника	DIV
Opcode	00100 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Арифметика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

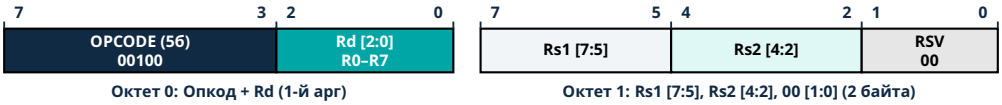
Назначение команды

Целочисленное деление с сохранением остатка во внутреннем регистре АЛУ.

Архитектурная семантика

```
Сигнатура:  DIV arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 / arg2
Операция:  arg0 <- arg1 / arg2
ALU_RESIDUAL <- arg1 % arg2
Пример:  DIV R0, R1, R2 ; R0 = R1 / R2 (остаток modulo в ALU_RESIDUAL)
```

Конфигурация и операнды



Октет 0: Опкод + Rd (1-й арг) — Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байта)

Октет 0: код операции 00100, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент, частное от деления).
Октет 1: биты $[7:5]$ задают делимое Rs1 (R0–R7); биты $[4:2]$ задают делитель Rs2 (R0–R7); биты $[1:0]$ содержат 00. Остаток от деления (modulo) автоматически фиксируется в служебном регистре ALU_RESIDUAL. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

При делении на ноль взводится аппаратный флаг ошибки EM (Error Math) без зависания ядра; обновляет ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).

3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Результат целочисленного деления (частное) записывается в целевой регистр dst, а остаток от деления (modulo) автоматически сохраняется во внутреннем служебном регистре АЛУ. Сохранённый остаток может быть задействован в последующих операциях АЛУ либо сброшен командой NOP.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

LMR

Загрузка данных из памяти и внешних устройств в регистры

00101
Память

Мнемоника	LMR
Opcode	00101 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Память
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

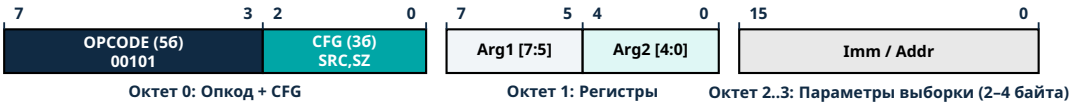
Назначение команды

Загрузка данных из памяти и внешних устройств в регистры.

Архитектурная семантика

```
Сигнатура:  LMR arg0(REG), <SRC_MODE> :  arg0 = MEM[...]
Режимы:  SRC=0:  LMR R0, [PC+] ; Imm Data (1-2 байта)
SRC=1:  LMR R0, [[PC+]] ; Imm Pointer
SRC=2:  LMR R0, [PTREG] ; Системный указатель (AIP)
SRC=3:  LMR R0(R_hi), R1(R_lo) ; 24-бит адрес из пары
Операция:  arg0 <- MEM[EA] (старшие биты адреса перезаписываются)
Пример:  LMR R0, R1 ; EA=(R0«16)|R1, чтение MEM[EA] в R0
```

Конфигурация и операнды



Октет 0: бит $[8]$ (SZ) задаёт разрядность ($0 = 1$ байт, $1 = 2$ байта / слово); биты $[10:9]$ (SRC) задают источник данных (0 — данные сразу за инструкцией в памяти, 1 — непосредственный указатель за инструкцией, 2 — косвенно по 24-битному указателю PTREG, 3 — косвенно по паре пользовательских регистров).
Октет 1: биты $[7:5]$ задают регистр Arg1 (R_hi / приёмник), биты $[4:2]$ задают регистр Arg2 (R_lo). В режиме SRC $= 3$ два 16-битных регистра формируют 32-битное число, из которого для адресации используются 24 бита: Arg1 хранит старшие биты адреса, а Arg2 — младшие 16 бит. При этом Arg1 одновременно является целевым регистром: после выборки из памяти старшие биты адреса в нём перезаписываются считанными данными.

Влияние на регистр флагов (FREG)

Все флаги регистра состояния FREG сохраняются без изменений.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Инструкция LMR является фундаментальным шлюзом ввода внешней информации в регистровый файл процессора LIM M2 ($R0$–$R7$). Архитектура поддерживает четыре ортогональных режима выборки и полную 24-битную адресацию:
1. В режиме SRC $= 0$ процессор считывает константу непосредственно из потока кода (1 или 2 байта) со смещением счетчика команд PC.
2. В режиме SRC $= 1$ за инструкцией считывается абсолютный указатель, по которому загружаются данные.
3. В режиме SRC $= 2$ чтение производится по 24-битному физическому адресу из системного регистра PTREG. При активном флаге автоинкремента FREG.AIP (бит 12) регистр PTREG автоматически инкрементируется на размер выборки ($+1$ или $+2$), обеспечивая аппаратный потоковый ввод массивов данных за один такт без накладных расходов.
4. В режиме SRC $= 3$ пользователь выбирает два 16-битных регистра (Arg1 и Arg2), которые вместе образуют 32-битное число, из которого на практике адресации используются младшие 24 бита. Пользователь указывает, что старшие биты адреса находятся в Arg1, а младшие 16 бит адреса — в Arg2. В этом режиме регистр Arg1 со старшими битами адреса также становится регистром назначения, куда сохраняются считанные данные (то есть старшие биты адреса в Arg1 перезаписываются данными, в то время как Arg2 сохраняет младшие биты адреса).

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

LRM

Запись данных из регистров в память и внешние устройства

00110
Память

Мнемоника	LRM
Opcode	00110 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Память
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

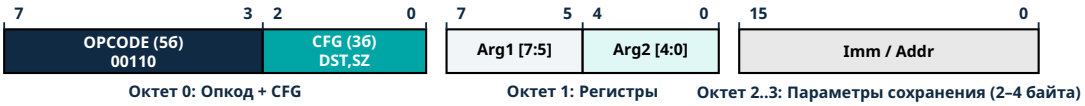
Назначение команды

Запись данных из регистров в память и внешние устройства.

Архитектурная семантика

```
Сигнатура:  LRM arg0(REG), <DST_MODE> :  MEM[...]  = arg0
Режимы:  DST=0:  LRM R0, [PC+] ; Inline память за кодом
DST=1:  LRM R0, [[PC+]] ; Imm Pointer
DST=2:  LRM R0, [PTREG] ; Системный указатель (AIP)
DST=3:  LRM R0(R_hi), R1(R_lo) ; 24-бит адрес из пары
Операция:  MEM[EA] <- arg0[15:0] (или младший байт при SZ=0)
Пример:  LRM R0, R1 ; EA=(R0«16)|R1, запись R0 в MEM[EA]
```

Конфигурация и операнды



Октет 0: бит $[8]$ (SZ) задаёт разрядность ($0 = 1$ байт, $1 = 2$ байта / слово); биты $[10:9]$ (DST) задают целевой приёмник данных (0 — запись сразу за кодом инструкции, 1 — запись по указателю за инструкцией, 2 — косвенно по 32-битному указателю PTREG, 3 — косвенно по паре регистров пользователя).
Октет 1: биты $[7:5]$ задают регистр-источник Arg1 (Rs / R_hi), биты $[4:2]$ задают регистр Arg2 (R_lo). В режиме DST $= 3$ два регистра формируют полный 32-битный физический адрес (Arg1 — старшие 16 бит адреса и источник данных, Arg2 — младшие 16 бит). Уничтожение существующих данных по адресу записи является ответственностью программиста.

Влияние на регистр флагов (FREG)

Все флаги регистра состояния FREG сохраняются без изменений.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Инструкция LRM является фундаментальным механизмом выгрузки регистровых данных процессора LIM M2 ($R0$–$R7$) в оперативную память и внешние порты периферии. Архитектура поддерживает четыре ортогональных режима адресации:
1. В режиме DST $= 0$ процессор записывает 1 или 2 байта непосредственно в память вслед за инструкцией (PC) со сдвигом счетчика команд. Режим применим для динамических буферов и самомодификации кода; перезапись данных лежит на ответственности программиста.
2. В режиме DST $= 1$ за инструкцией считывается 16-битный абсолютный адрес, по которому сохраняются данные.
3. В режиме DST $= 2$ запись производится по 32-битному системному адресу регистра PTREG. При активном флаге FREG . AIP (бит 12) регистр PTREG автоматически инкрементируется на $+1$ или $+2$, обеспечивая аппаратный потоковый DMA-вывод массивов в память без программных циклов.
4. В режиме DST $= 3$ используется пара 16-битных регистров: Arg1 и Arg2 образуют полный 32-битный составной физический адрес (Addr$[31:0] = (\texttt{Arg1} \ll 16) \mid \texttt{Arg2}$). Данные из Arg1 записываются по вычисленному адресу, обеспечивая прямой доступ ко всему адресному пространству 4 ГБ.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

LRR

Межрегистровое перемещение (копирование из точки Б в точку А)

00111
Пересылка

Мнемоника	LRR
Opcode	00111 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Пересылка
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

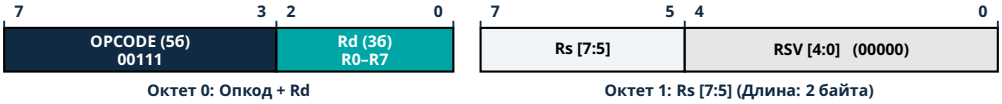
Назначение команды

Межрегистровое перемещение (копирование из точки Б в точку А).

Архитектурная семантика

```
Сигнатура:  LRR arg0(REG), arg1(REG) : arg0 = arg1
Операция:  arg0 <- arg1 (прямое копирование из точки Б в точку А)
Пример:  LRR R0, R1 ; R0 = R1 (значение R1 копируется в R0)
```

Конфигурация и операнды



Октет 0: код операции 00111, биты $[2:0]$ задают целевой регистр Rd (R0–R7, точка А, куда сохраняются данные, первый аргумент).
Октет 1: биты $[7:5]$ задают регистр-источник Rs (R0–R7, точка Б, откуда берутся данные); биты $[4:0]$ содержат 00000_2. При исполнении значение Rs передаётся в Rd за 1 микротакт.

Влияние на регистр флагов (FREG)

Все флаги регистра состояния FREG сохраняются без изменений.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Инструкция LRR является фундаментальной операцией пересылки данных между регистрами общего назначения процессора ($R0$–$R7$). Она реализует передачу из источника (точка Б) в приёмник (точка А). Пересылка выполняется через прямой аппаратный обходной тракт **MUX 1** $\rightarrow$ **MUX 2** минуя АЛУ, что исключает блокировку вычислительного тракта и не влияет на флаги регистра FREG.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

CMP

Аппаратное сравнение операндов

01000
Сравнение

Мнемоника	CMP
Opcode	01000 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Сравнение
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

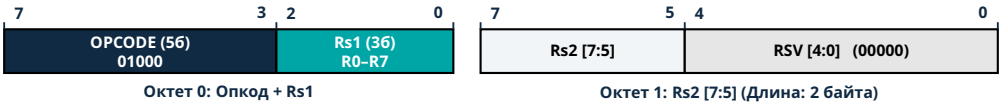
Назначение команды

Аппаратное сравнение операндов.

Архитектурная семантика

```
Сигнатура:  CMP arg0(REG), arg1(REG) : compare(arg0, arg1)
Операция:  if arg0 < arg1 then FREG.CMP <- 01b (меньше)
if arg0 == arg1 then FREG.CMP <- 10b (равно)
if arg0 > arg1 then FREG.CMP <- 11b (больше)
(состояние 00b — сброс / не сравнивалось / ошибка)
Пример:  CMP R0, R1 ; 01:  R0<R1, 10:  R0==R1, 11:  R0>R1 (00:  сброс/ошибка)
```

Конфигурация и операнды



Октет 0: код операции 01000, биты $[2:0]$ задают первый операнд Rs1 (первый аргумент).
Октет 1: биты $[7:5]$ задают второй операнд Rs2; биты $[4:0]$ содержат 00000.
Результат аппаратного сравнения фиксируется в поле FREG.CMP (биты $[5:4]$): 01 (Rs1 $<$ Rs2), 10 (Rs1 $=$ Rs2), 11 (Rs1 $>$ Rs2), 00 (сброс / не сравнивалось / ошибка). Значения регистров не модифицируются. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет двухбитное поле FREG.CMP (биты [5:4]), а также флаги ZF, CF, OF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).

3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Позволяет выполнять быстрые условные переходы без искажения регистров.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

JMP

Безусловный переход по адресу

01001
Переход

Мнемоника	JMP
Opcode	01001 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Переход
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Безусловный переход по адресу.

Архитектурная семантика

```
Сигнатура:  JMP target(IMM|REG) : PC = target
Операция:  PC <- target
Пример:  JMP R0 ; Безусловный переход по адресу в R0
```

Конфигурация и операнды



Октет 0: код операции 01001, биты $[2:0]$ (MOD): 000 — 16-битный непосредственный адрес (длина 3 байта, октеты 1–2 содержат адрес); 001 — косвенный регистровый переход (длина 2 байта, октет 1 биты $[7:5]$ задают целевой регистр Rs); 010 — расширенный 24-битный переход по системному указателю PTREG.

Влияние на регистр флагов (FREG)

Флаги сохраняются без изменений.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Атомарная загрузка Program Counter за минимальное число микротактов.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

JCR

Условный переход по флагу переноса

01010
Переход

Мнемоника	JCR
Opcode	01010 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Переход
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Условный переход по флагу переноса.

Архитектурная семантика

```
Сигнатура:  JCR target(IMM|REG) : if FREG.CF == 1 then PC = target
Операция:  if FREG.CF == 1 then PC <- target
Пример:  JCR R2 ; Переход по адресу R2 при наличии переноса
```

Конфигурация и операнды



Октет 0: код операции 01010, биты $[2:0]$ (MOD): 000 (16-бит адрес Imm, 3 байта), 001 (косвенно по Rs, 2 байта), 010 (24-бит по PTREG). Переход осуществляется строго при установленном флаге переноса FREG.CF = 1.

Влияние на регистр флагов (FREG)

Флаги читаются, но не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Ключевой примитив для построения длинной многословной арифметики.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

JNZ

Условный переход при неравенстве нулю

01011
Переход

Мнемоника	JNZ
Opcode	01011 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Переход
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Условный переход при неравенстве нулю.

Архитектурная семантика

```
Сигнатура:  JNZ target(IMM|REG) : if FREG.ZF == 0 then PC = target
Операция:  if FREG.ZF == 0 then PC <- target
Пример:  JNZ R3 ; Переход по адресу R3 при ZF == 0 (не ноль)
```

Конфигурация и операнды



Октет 0: код операции 01011, биты $[2:0]$ (MOD): 000 (16-бит адрес Imm, 3 байта), 001 (косвенно по Rs, 2 байта), 010 (24-бит по PTREG). Переход выполняется при сброшенном флаге нуля FREG.ZF = 0.

Влияние на регистр флагов (FREG)

Флаги не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Высокоэффективный переход для организации циклов и счетчиков.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

JZ

Условный переход при равенстве нулю

01100
Переход

Мнемоника	`JZ`
Opcode	`01100` (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Переход
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Условный переход при равенстве нулю.

Архитектурная семантика

```
Сигнатура:  JZ target(IMM|REG) : if FREG.ZF == 1 then PC = target
Операция:  if FREG.ZF == 1 then PC <- target
Пример:  JZ R4 ; Переход по адресу R4 при ZF == 1 (равно 0)
```

Конфигурация и операнды



Октет 0: код операции `01100`, биты $[2:0]$ (MOD): `000` (16-бит адрес Imm, 3 байта), `001` (косвенно по Rs, 2 байта), `010` (24-бит по PTREG). Переход выполняется при взведённом флаге нуля FREG.ZF = 1.

Влияние на регистр флагов (FREG)

Флаги не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Оптимизирован для алгоритмов поиска и сравнения строк.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

AND

Побитовое логическое И

01101
Логика

Мнемоника	AND
Opcode	01101 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Логика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

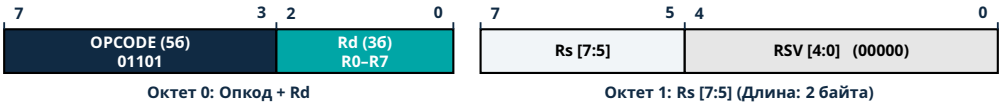
Назначение команды

Побитовое логическое И.

Архитектурная семантика

```
Сигнатура:  AND arg0(REG), arg1(REG) : arg0 = arg0 & arg1
Операция:  arg0 <- arg0 AND arg1
Пример:  AND R0, R1 ; R0 = R0 & R1
```

Конфигурация и операнды



Октет 0: Опкод + Rd

Октет 1: Rs [7:5] (Длина: 2 байта)

Октет 0: код операции 01101, биты $[2:0]$ прямо задают целевой регистр Rd (R0–R7, первый аргумент, Rd $\leftarrow$ Rd $\&$ Rs).
Октет 1: биты $[7:5]$ задают второй операнд Rs (R0–R7); биты $[4:0]$ содержат 00000_2. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет флаг нуля ZF; сбрасывает флаги переполнения.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Идеально для наложения битовых масок.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

OR

Побитовое логическое ИЛИ

01110
Логика

Мнемоника	OR
Opcode	01110 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Логика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

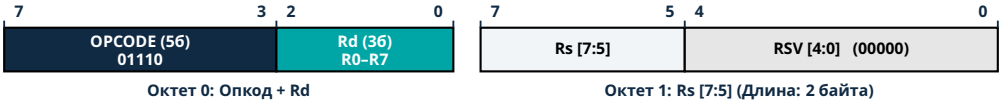
Назначение команды

Побитовое логическое ИЛИ.

Архитектурная семантика

```
Сигнатура:  OR arg0(REG), arg1(REG) : arg0 = arg0 | arg1
Операция:  arg0 <- arg0 OR arg1
Пример:  OR R0, R1 ; R0 = R0 | R1
```

Конфигурация и операнды



Октет 0: код операции 01110, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент, Rd $\leftarrow$ Rd | Rs).
Октет 1: биты $[7:5]$ задают второй операнд Rs (R0–R7); биты $[4:0]$ содержат 00000_2. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF; флаги переноса детерминированы.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Быстрое объединение управляющих битов.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

NAND

Побитовое логическое И-НЕ

01111
Логика

Мнемоника	NAND
Opcode	01111 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Логика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

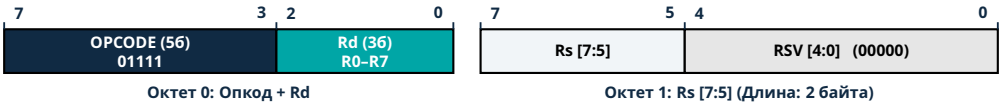
Назначение команды

Побитовое логическое И-НЕ.

Архитектурная семантика

```
Сигнатура:  NAND arg0(REG), arg1(REG) : arg0 = ~(arg0 & arg1)
Операция:  arg0 <- NOT (arg0 AND arg1)
Пример:  NAND R0, R1 ; R0 = ~(R0 & R1)
```

Конфигурация и операнды



Октет 0: Опкод + Rd — Октет 1: Rs [7:5] (Длина: 2 байта)

Октет 0: код операции 01111, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент).
Октет 1: биты $[7:5]$ задают операнд Rs (R0–R7); биты $[4:0]$ содержат 00000_2. Длина: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Позволяет сократить код при синтезе нестандартных логических функций.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

NOR

Побитовое логическое ИЛИ-НЕ

10000
Логика

Мнемоника	NOR
Opcode	10000 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Логика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

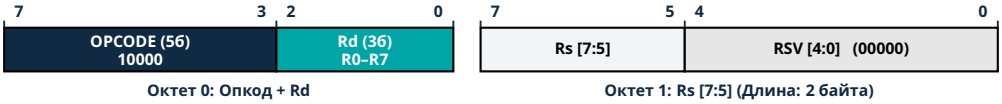
Назначение команды

Побитовое логическое ИЛИ-НЕ.

Архитектурная семантика

```
Сигнатура:  NOR arg0(REG), arg1(REG) : arg0 = ~(arg0 | arg1)
Операция:  arg0 <- NOT (arg0 OR arg1)
Пример:  NOR R0, R1 ; R0 = ~(R0 | R1)
```

Конфигурация и операнды



Октет 0: код операции 10000, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент).
Октет 1: биты $[7:5]$ задают операнд Rs (R0–R7); биты $[4:0]$ содержат 00000_2. Длина: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Расширяет набор быстрых логических трансформаций.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

NOT

Побитовое инвертирование (дополнение до 1)

10001
Логика

Мнемоника	NOT
Opcode	10001 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Логика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Побитовое инвертирование (дополнение до 1).

Архитектурная семантика

```
Сигнатура:  NOT arg0(REG) : arg0 = ~arg0
Операция:  arg0 <- NOT arg0
Пример:  NOT R0 ; R0 = ~R0 (1-байтовая инструкция)
```

Конфигурация и операнды



Октет 0 (1 байт, 0 доп. байт)

Однобайтовая инструкция (длина: 1 байт). Октет 0: биты $[7:3]$ содержат опкод 10001, биты $[2:0]$ прямо задают целевой регистр Rd (R0–R7). Второй байт из шины памяти не запрашивается.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Выполняется за минимальное количество микротактов без чтения второго байта.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

XOR

Побитое исключающее ИЛИ

10010
Логика

Мнемоника	XOR
Opcode	10010 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Логика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Побитое исключающее ИЛИ.

Архитектурная семантика

```
Сигнатура:  XOR arg0(REG), arg1(REG) : arg0 = arg0 ^ arg1
Операция:  arg0 <- arg0 XOR arg1
Пример:  XOR R0, R1 ; R0 = R0 ^ R1 (XOR R0, R0 обнуляет R0)
```

Конфигурация и операнды



Октет 0: код операции 10010, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент, Rd $\leftarrow$ Rd $\oplus$ Rs).
Октет 1: биты $[7:5]$ задают второй операнд Rs (R0–R7); биты $[4:0]$ содержат 00000_2. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Базовая операция для алгоритмов контрольных сумм и шифрования.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

XNOR

Побитовая эквивалентность

10011
Логика

Мнемоника	XNOR
Opcode	10011 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Логика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

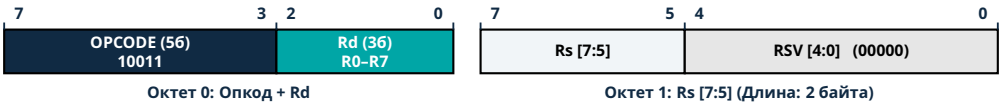
Назначение команды

Побитовая эквивалентность.

Архитектурная семантика

```
Сигнатура:  XNOR arg0(REG), arg1(REG) : arg0 = ~(arg0 ^ arg1)
Операция:  arg0 <- NOT (arg0 XOR arg1)
Пример:  XNOR R0, R1 ; R0 = ~(R0 ^ R1)
```

Конфигурация и операнды



Октет 0: код операции 10011, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент).
Октет 1: биты $[7:5]$ задают операнд Rs (R0–R7); биты $[4:0]$ содержат 00000_2. Длина: 2 байта.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Эффективная проверка соответствия битовых масок.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

INC

Инкремент операнда на единицу

10100
Арифметика

Мнемоника	INC
Opcode	10100 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Арифметика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Инкремент операнда на единицу.

Архитектурная семантика

```
Сигнатура:  INC arg0(REG) : arg0 = arg0 + 1
Операция:  arg0 <- arg0 + 1
Пример:  INC R0 ; R0 = R0 + 1 (1-байтовая инструкция)
```

Конфигурация и операнды



Октет 0 (1 байт, 0 доп. байт)

Однобайтовая инструкция (длина: 1 байт). Октет 0: биты $[7:3]$ содержат опкод 10100, биты $[2:0]$ прямо задают регистр Rd (R0–R7). Второй байт из шины памяти не запрашивается.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF. Переполнение отслеживается АЛУ.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Быстрый шаг итератора цикла за минимальное количество тактов.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

DEC

Декремент операнда на единицу

10101
Арифметика

Мнемоника	DEC
Opcode	10101 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Арифметика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Декремент операнда на единицу.

Архитектурная семантика

```
Сигнатура:  DEC arg0(REG) : arg0 = arg0 - 1
Операция:  arg0 <- arg0 - 1
Пример:  DEC R0 ; R0 = R0 - 1 (1-байтовая инструкция)
```

Конфигурация и операнды



Октет 0 (1 байт, 0 доп. байт)

Однобайтовая инструкция (длина: 1 байт). Октет 0: биты $[7:3]$ содержат опкод 10101, биты $[2:0]$ прямо задают регистр Rd (R0–R7). Второй байт из шины памяти не запрашивается.

Влияние на регистр флагов (FREG)

Обновляет флаг ZF. Заём отслеживается АЛУ.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Компактная реализация декрементных циклов без чтения второго байта.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

PUSH

Сохранение слова в стек

10110
Стек

Мнемоника	PUSH
Opcode	10110 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Стек
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

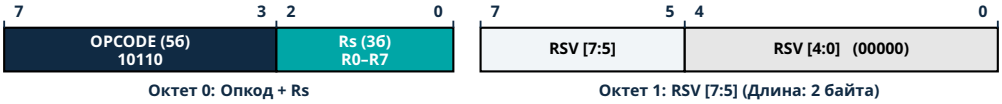
Назначение команды

Сохранение слова в стек.

Архитектурная семантика

```
Сигнатура:  PUSH arg0(REG) : SREG -= 2, MEM[SREG] = arg0
Операция:  SREG <- SREG - 2, MEM[SREG] <- arg0
Пример:  PUSH R0 ; Сохранить слово R0 в вершину стека
```

Конфигурация и операнды



Октет 0: Опкод + Rs

Октет 1: RSV [7:5] (Длина: 2 байта)

Октет 0: код операции 10110, биты $[2:0]$ прямо задают сохраняемый регистр-источник Rs (R0–R7, первый аргумент).
Октет 1: биты $[7:0]$ содержат 00000000_2 (резерв). Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Флаги не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Аппаратная поддержка вызова процедур и сохранения контекста.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

POP

Извлечение слова из вершины стека

10111
Стек

Мнемоника	POP
Opcode	10111 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Стек
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

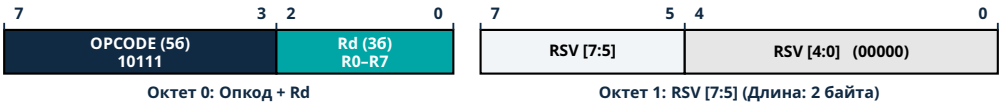
Назначение команды

Извлечение слова из вершины стека.

Архитектурная семантика

```
Сигнатура:  POP arg0(REG) : arg0 = MEM[SREG], SREG += 2
Операция:  arg0 <- MEM[SREG], SREG <- SREG + 2
Пример:  POP R0 ; Извлечь слово из вершины стека в R0
```

Конфигурация и операнды



Октет 0: Опкод + Rd — Октет 1: RSV [7:5] (Длина: 2 байта)

Октет 0: код операции 10111, биты $[2:0]$ прямо задают целевой регистр-приёмник Rd (R0–R7, первый аргумент).
Октет 1: биты $[7:0]$ содержат 00000000_2 (резерв). Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Флаги не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Симметричное восстановление контекста прерываний и функций.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

BSL

Аппаратный сдвиг влево (итеративный регистр)

11000
Сдвиги

Мнемоника	BSL
Opcode	11000 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Сдвиги
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

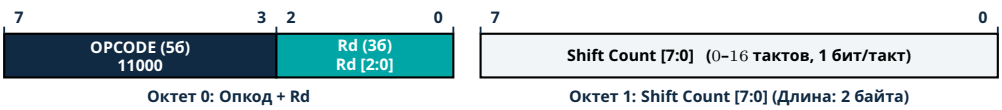
Назначение команды

Аппаратный сдвиг влево (итеративный регистр).

Архитектурная семантика

```
Сигнатура:  BSL arg0(REG), arg1(IMM) : arg0 = arg0 « arg1
Кодирование:  Октет 0:  [7:3]=11000, [2:0]=Rd (целевой регистр R0-R7)
              Октет 1:  [7:0]=shift (число битов сдвига 0-16)
Операция:  Последовательный тактовый сдвиг регистра Rd влево на shift бит:
Аппаратный цикл (1 такт на 1 бит, ≈ 300 транзисторов):
Для каждого такта t ∈ [1..shift]:
Старший бит 15 вытесняется (last_displaced <- bit 15);
При SMOD = 1 в младший бит 0 влетает вытесненный бит (ротация);
При SMOD = 0 в младший бит 0 влетает бит из FREG.DBIT (0 или 1);
Фиксация:  FREG.DBIT <- last_displaced (последний выброшенный бит).
Пример:  BSL R0, 4 ; Сдвиг R0 влево на 4 бита (занимает 4 такта сдвига)
```

Конфигурация и операнды



Октет 0: код операции 11000, биты $[2:0]$ прямо задают сдвигаемый регистр Rd (R0–R7).
Октет 1: весь байт ([7:0]) указывает число тактов сдвига shift (0–16). Физически сдвиг реализуется потактовым циклом на 16 D-триггерах. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Последний вытесненный бит сохраняется в FREG.DBIT (бит 13). Флаг нуля FREG.ZF устанавливается при равенстве результата нулю. Поведение определяется FREG.SMOD (бит 14) и FREG.DBIT (бит 13).

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Итеративный сдвиговый регистр на D-триггерах: стоимость всего ≈ 300 транзисторов вместо $\approx 30\,000$ для матричного однотактового сдвигателя. Сдвиг на 16 бит требует на 15 тактов больше, чем сдвиг на 1 бит.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

BSR

Аппаратный сдвиг вправо (итеративный регистр)

11001
Сдвиги

Мнемоника	BSR
Opcode	11001 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Сдвиги
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

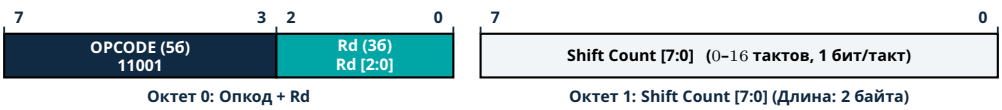
Назначение команды

Аппаратный сдвиг вправо (итеративный регистр).

Архитектурная семантика

```
Сигнатура:  BSR arg0(REG), arg1(IMM) : arg0 = arg0 » arg1
Кодирование:  Октет 0:  [7:3]=11001, [2:0]=Rd (целевой регистр R0-R7)
              Октет 1:  [7:0]=shift (число битов сдвига 0-16)
Операция:  Последовательный тактовый сдвиг регистра Rd вправо на shift бит:
Аппаратный цикл (1 такт на 1 бит, ≈ 300 транзисторов):
Для каждого такта t ∈ [1..shift]:
Младший бит 0 вытесняется (last_displaced <- bit 0);
При SMOD = 1 в старший бит 15 влетает вытесненный бит (ротация);
При SMOD = 0 в старший бит 15 влетает бит из FREG.DBIT (0 или 1);
Фиксация:  FREG.DBIT <- last_displaced (последний выброшенный бит).
Пример:  BSR R0, 4 ; Сдвиг R0 вправо на 4 бита (занимает 4 такта сдвига)
```

Конфигурация и операнды



Октет 0: код операции 11001, биты $[2:0]$ прямо задают сдвигаемый регистр Rd (R0–R7).
Октет 1: весь байт ([7:0]) указывает число тактов сдвига shift (0–16). Физически сдвиг реализуется потактовым циклом на 16 D-триггерах. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Последний вытесненный бит сохраняется в FREG.DBIT (бит 13). Флаг нуля FREG.ZF устанавливается при равенстве результата нулю. Поведение определяется FREG.SMOD (бит 14) и FREG.DBIT (бит 13).

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Итеративный сдвиговый регистр на D-триггерах: стоимость всего ≈ 300 транзисторов вместо $\approx 30\,000$ для матричного однотактового сдвигателя. Сдвиг на 16 бит требует на 15 тактов больше, чем сдвиг на 1 бит.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

MSP

Манипуляция спецрегистрами (FREG, PTREG, SREG, VEC)

11010
Система

Мнемоника	MSP
Opcode	11010 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Система
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

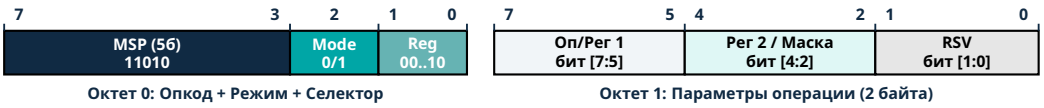
Назначение команды

Манипуляция спецрегистрами (FREG, PTREG, SREG, VEC).

Архитектурная семантика

```
Сигнатура:  MSP sreg, [op], args :  работа со спец-регистрами (2–6 байт)
Операция:  Прямая загрузка (бит 10 = 0):
sreg=00 (FREG): FREG <- Rs[15:0] или imm16 / [addr16] / [$ptreg]
sreg=01 (PTREG): 32-битный указатель PTREG (4 ГБ): из POH, imm32, [$ptreg],
[addr32]
sreg=10 (SREG): 32-битный указатель стека SREG: из POH, imm32, [$ptreg],
[addr32]
sreg=11 (VEC): 32-битный вектор прерывания ($ir0..$ir15) или зоны
($pv0..$pv15)
Модификация (бит 10 = 1):
sreg=00 (FREG): FREG <- FREG ⊙ Mask {AND, OR, XOR, XNOR}
sreg=01 (PTREG): PTREG <- PTREG ± Offset {ADD, SUB}
sreg=10 (SREG): SREG <- SREG ± Offset {ADD, SUB}
Пример:  MSP.ld $ir0, 0x00010000 ; Установка вектора IRQ0 (6 байт)
MSP.ld $pv0, 0x00008000 ; Начало защищенной Зоны Привилегий 0
MSP.ld $pv1, 0x00008FFF ; Конец защищенной Зоны Привилегий 0
MSP.ld $ptreg, [$ptreg] ; Нативное 1-тактовое разыменование
MSP.or $freg, 0x0200 ; Включение глобальных прерываний MIE
```

Конфигурация и операнды



Октет 0: код операции 11010. Бит $[10]$ (бит 2 октета 0) задаёт режим: 0 — прямая загрузка, 1 — модификация. Биты $[9:8]$ (биты 1–0 октета 0) задают селектор: 00 = FREG, 01 = PTREG, 10 =

SREG.
Октет 1: при загрузке ($[10] = 0$) задаёт источник: Rs для FREG или пару Rs_hi, Rs_lo для 24-битных регистров PTREG и SREG. При модификации ($[10] = 1$) биты $[7:6]$ кодируют операцию (для FREG: AND, OR, XOR, XNOR; для PTREG и SREG: ADD, SUB), а биты $[5:3]$ задают ровно один регистр операнда/маски Rs. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

Флаги FREG модифицируются напрямую либо через маски. Арифметика PTREG и SREG формирует флаг переноса CF через АЛУ.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Универсальный управляющий примитив системного уровня для работы с флагами, указателем памяти и указателем стека.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

MOD

Целочисленный остаток от деления (modulo)

11011
Арифметика

Мнемоника	MOD
Opcode	11011 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Арифметика
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

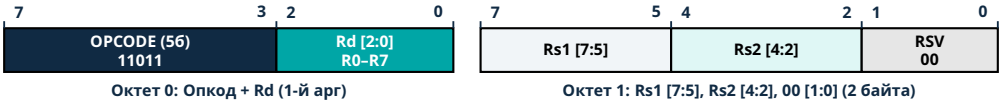
Назначение команды

Целочисленный остаток от деления (modulo).

Архитектурная семантика

```
Сигнатура:  MOD arg0, arg1, arg2 :  arg0 = arg1 % arg2
Операция:  dst <- arg1 % arg2
Пример:  MOD R0, R1, R2 ; R0 = R1 % R2 (остаток от деления)
```

Конфигурация и операнды



Октет 0: код операции 11011, биты $[2:0]$ задают целевой регистр Rd (R0–R7, первый аргумент, получатель остатка).
Октет 1: биты $[7:5]$ задают делимое Rs1 (R0–R7); биты $[4:2]$ задают делитель Rs2 (R0–R7); биты $[1:0]$ содержат 00. Длина команды: 2 байта.

Влияние на регистр флагов (FREG)

При делении на ноль взводится аппаратный флаг ошибки EM (Error Math) без зависания ядра; обновляет ZF.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Вычисляет остаток от целочисленного деления делимого Rs1 на делитель Rs2 и сохраняет результат непосредственно в целевой регистр Rd, не перезаписывая промежуточные результаты в аккумуляторах.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

INTR

Программное прерывание по вектору из стека векторов ядра

11100
Система

Мнемоника	INTR
Opcode	11100 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Система
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Программное прерывание по вектору из стека векторов ядра.

Архитектурная семантика

```
Сигнатура:  INTR vec(IMM4) / INT vec :  вызов вектора прерывания 0..15
Операция:  SREG <- SREG - 4, MEM32[SREG] <- PC
SREG <- SREG - 2, MEM16[SREG] <- FREG
FREG.INTR <- 1, FREG.PRV <- 1, FREG.MIE <- 0, PC <- VECTORS[vec]
Пример:  INT 5 ; Программное прерывание по вектору 5 (2 байта)
```

Конфигурация и операнды



Двухбайтовая инструкция (длина: 2 байта). Октет 0: биты $[7:3]$ содержат опкод 11100, младшие 3 бита $[2:0]$ содержат 000. Октет 1: старшие 4 бита $[7:4]$ зарезервированы (0000), а младшие 4 бита $[3:0]$ задают номер вектора vec (0–15) из таблицы 16 векторов прерываний ($ir0..$ir15).

Влияние на регистр флагов (FREG)

Устанавливает флаги FREG.INTR = 1 (бит 15), FREG.PRV = 1 (бит 11, режим супервизора) и сбрасывает маску прерываний FREG.MIE = 0 (бит 9); исходные 32-битный PC и 16-битный FREG сохраняются в стек SREG.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память /

переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Обеспечивает защищённый программный вызов системных обработчиков, системных вызовов ОС (Syscall) и программных ловушек по номеру вектора 0..15.

CALL

Вызов подпрограммы с сохранением PC в стек

11101
Переходы

Мнемоника	CALL
Opcode	11101 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Переходы
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Вызов подпрограммы с сохранением PC в стек.

Архитектурная семантика

```
Сигнатура:  CALL target :  SREG -= 4, MEM32[SREG] = PC, PC = target
Кодирование:  Октет 0:  [7:3]=11101, [2:0]=cfg (0:  imm16, 1:  Rd, 2:
[$ptreg], 3:  [addr16])
Операция:  ret_addr <- PC
SREG <- SREG - 4, MEM32[SREG] <- ret_addr
PC <- target
Пример:  CALL func ; Вызов подпрограммы по метке/адресу func (3 байта)
CALL $0 ; Косвенный вызов через регистр R0 (2 байта)
CALL [$ptreg] ; Вызов по 32-битному указателю PTREG (2 байта)
```

Конфигурация и операнды



Октет 0: код операции 11101, биты $[2:0]$ задают режим адресации цели перехода (cfg): 0 — прямой 16-битный адрес target (3 байта), 1 — косвенно через РОН Rd (2 байта), 2 — через 32-битный указатель PTREG (2 байта), 3 — косвенно по адресу в памяти (3 байта).
Перед совершением перехода полный 32-битный адрес возврата (адрес следующей инструкции) сохраняется в вершину системного стека SREG.

Влияние на регистр флагов (FREG)

Флаги не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.

2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Стандартная инструкция вызова процедур и функций с сохранением полного 32-битного контекста адреса возврата в стеке SREG.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

RET

Возврат из подпрограммы с извлечением PC из стека

11110
Переходы

Мнемоника	RET
Opcode	11110 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Переходы
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Возврат из подпрограммы с извлечением PC из стека.

Архитектурная семантика

```
Сигнатура:  RET : PC = MEM32[SREG], SREG += 4
Операция:  PC <- MEM32[SREG], SREG <- SREG + 4
Пример:  RET ; Возврат из подпрограммы (1 байт)
```

Конфигурация и операнды



Октет 0 (1 байт, 0 доп. байт)

Однобайтовая инструкция (длина: 1 байт). Октет 0: код операции 11110, биты $[2:0]$ содержат 000. Восстанавливает 32-битный программный счётчик PC из вершины стека SREG и освобождает 4 байта стека (SREG += 4).

Влияние на регистр флагов (FREG)

Флаги не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

Особенности реализации и примечания

Симметричная инструкция возврата из подпрограмм, вызванных командой CALL.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

HALT

Аппаратный останов процессора

11111
Управление

Мнемоника	HALT
Opcode	11111 (старшие 5 бит кода команды)
Формат команды	5 бит opcode, 3 бита cfg и аргументы переменного формата (у специализированных команд — операнды сразу после opcode)
Класс операции	Управление
Статус фиксации	Базовый функционал утверждён; битовые поля аргументов и расширенные модификаторы находятся на стадии финализации

Назначение команды

Аппаратный останов процессора.

Архитектурная семантика

```
Сигнатура:  HALT / HLT : останов тактирования ядра
Операция:  CPU.HALTED <- 1, PINS.HALT <- 1
Пример:  HALT ; Полный останов процессора (1 байт)
```

Конфигурация и операнды



Октет 0 (1 байт, 0 доп. байт)

Однобайтовая инструкция (длина: 1 байт). Октет 0: код операции 11111, биты $[2:0]$ содержат `000`. Переводит процессор в режим останова ядра до подачи аппаратного сигнала сброса RESET или внешнего прерывания IRQ.

Влияние на регистр флагов (FREG)

Флаги не изменяются.

Микрооперации исполнительного тракта

1. Коммутация мультиплексоров тракта данных для выборки исходных операндов.
2. Исполнение операции в специализированном функциональном блоке (АЛУ / память / переходы).
3. Направление результата через арбитр записи (MUX 2) к целевому регистру-приёмнику.
4. Синхронное обновление аппаратных флагов FREG и служебных регистров (PTREG, SREG).

Точная сетка микротактов и управляющие стробы: **TBD**.

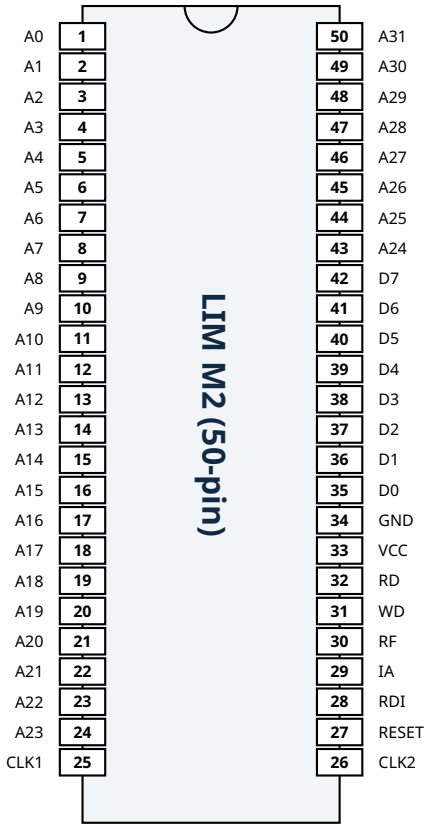
Особенности реализации и примечания

Завершение работы процессора или переход в режим энергосбережения.

Параметры с отметкой TBD соответствуют расширенным опциям спецификации.

7 Физический интерфейс и цоколёвка (50-выводный корпус)

Процессор размещается в 50-выводном корпусе DIP-50 / QFP-52 с нативной 32-разрядной шиной адреса и 8-битной (однобайтовой) внешней шиной данных:



7.1 Функциональное назначение внешних линий

Вывод	Тип	Описание системного сигнала
A0–A31	Выход / Tri-State	32-разрядная нативная магистраль адреса (прямой физический доступ к 4 ГБ памяти без окон и банков).
D0–D7	Двунаправленный	8-разрядная (однобайтовая) внешняя шина данных обмена с памятью и периферией.
CLK1, CLK2	Вход	Двухфазная тактовая синхронизация с защитой от перекрытия фаз.
RESET	Вход	Аппаратный сигнал сброса процессора (активный низкий).
RDI	Вход	Raw Direct Interrupt: приоритетный вход аппаратного прерывания.
IA	Выход	Interruption Accepted / Interrupt Acknowledge: строб подтверждения прерывания устройству.
RF	Вход	Ready Flag: аппаратная квитанция готовности ведомого устройства памяти/периферии.
RD, WD	Выход	Стробы чтения (Read Enable) и записи (Write Enable) системной шины.
VCC, GND	Питание	Основные шины цифрового питания (+3.3 В / +5 В номинал) и земля.

8 Подсистема прерываний и векторная обработка

Подсистема прерываний процессора LIM M2 спроектирована для обеспечения минимальной латентности реакции на события в реальном времени, строгого детерминизма и полной защиты от шинных конфликтов. Архитектура объединяет аппаратные внешние запросы и программные исключения в единую 16-векторную иерархию с надёжным аппаратным квитированием на физическом уровне.

8.1 Единое пространство 16 векторов прерываний

Процессор поддерживает ровно 16 системных векторов (номера 0–15), которые являются ортогональными и общими как для внешней периферии, так и для внутренних событий ядра:

Table 3: Единая таблица 16 векторов прерываний LIM M2

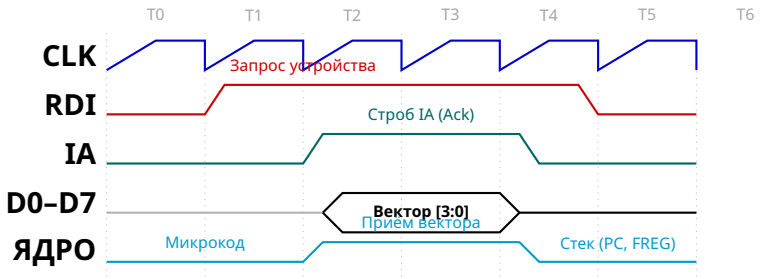
Вектор	Код	Источник / Тип	Функциональное назначение
VEC 0	`0x00`	Аппаратный / Сброс	Power-on Reset: начальная инициализация процессора и регистров.
VEC 1	`0x01`	Аппаратный (NMI)	Немаскируемое прерывание: сбой питания, аппаратная авария системы.
VEC 2	`0x02`	Программная ловушка	Арифметическое исключение (деление на ноль, флаг `FREG.EM`).
VEC 3	`0x03`	Программная ловушка	Системный вызов ядра (System Call / Trap диспетчера ОС).
VEC 4	`0x04`	Аппаратный / Периферия	Внешний таймер / Системный квантователь времени.
VEC 5	`0x05`	Аппаратный / Периферия	Контроллер последовательного интерфейса (UART RX/TX).
VEC 6	`0x06`	Аппаратный / Периферия	Контроллер прямого доступа к памяти (DMA Ready).
VEC 7	`0x07`	Аппаратный / Периферия	Внешний контроллер прерываний / Шинный мост.
VEC 8–15	`0x08–0x0F`	Общие (HW / SW)	Пользовательские аппаратные линии и программные векторы.

8.2 Аппаратный протокол квитирования (RDI → IA → D0–D7)

Для взаимодействия с внешними контроллерами реализован надёжный 4-фазный цикл аппаратного рукопожатия (handshake), исключающий необходимость предварительного опроса регистров:

1. **Фаза запроса (RDI):** Периферийное устройство выставляет высокий логический уровень на входе `RDI` (*Raw Direct Interrupt*). Линия опрашивается ядром на границах микрокоманд.
2. **Фаза проверки маски (Арбитраж):** Если глобальное маскирование прерываний разрешено (бит `FREG.MIE = 1`), процессор фиксирует запрос и завершает текущую неделимую микрооперацию тракта данных.
3. **Фаза подтверждения (Строб IA):** Ядро генерирует активный строб на выходе `IA` (*Interruption Accepted / Interrupt Acknowledge*, вывод 26). Этот строб извещает все внешние устройства на шине о готовности ядра принять номер вектора.
4. **Фаза передачи вектора (Шина D0–D7):** Одновременно с выдачей `IA` внешняя шина данных `D0–D7` переводится ядром в режим приёма (вход), а линия чтения `RD` активируется. Запросившее устройство удерживает на линиях `D0–D7` 8-битный идентификатор вектора (младшие 4 бита адресуют векторы 0–15).

5. **Фаза фиксации и переход:** Процессор защёлкивает данные с шины D0–D7, снимает сигнал IA и переходит к процедуре сохранения контекста.



8.3 Сохранение контекста в 24-битном стеке и вход в обработчик

Принятие прерывания разрешено строго при условии, что процессор не находится в состоянии обработки другого прерывания (FREG.INTR = 0) и прерывания глобально разрешены (FREG.MIE = 1). Если FREG.INTR = 1, любые повторные аппаратные прерывания аппаратно заблокированы.

При фиксации прерывания ядро выполняет следующую аппаратную процедуру:

1. **Сохранение адреса возврата (PC):** Текущее значение счётчика команд PC сохраняется на вершине системного стека со смещением 24-битного указателя SREG:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{PC}_{15:0}$$

2. **Сохранение флагов и статуса (FREG):** Текущее значение регистра FREG (с исходным значением FREG.INTR=0) помещается в стек:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{FREG}$$

3. **Аппаратная блокировка вложенных прерываний:** Ядро выставляет флаг нахождения в прерывании FREG.INTR $\leftarrow$ 1 (бит 15) и сбрасывает маску FREG.MIE $\leftarrow$ 0. Наличие активного бита FREG.INTR гарантирует невозможность каскадных или паразитных повторных входов в обработчики.
4. **Переход по вектору:** Базовый адрес обработчика вычисляется на основе принятого номера вектора и загружается в PC.

8.4 Возврат из прерывания

Выход из процедуры обработки прерывания осуществляется восстановлением сохранённого контекста со стека:

1. Со стека считывается и восстанавливается исходное состояние регистра флагов:

$$\text{FREG} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

   (При этом бит FREG.INTR автоматически возвращается в 0, снимая блокировку прерываний, а бит FREG.MIE восстанавливает своё исходное значение).
2. Со стека восстанавливается счётчик команд:

$$\text{PC} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

3. Процессор возобновляет выполнение прерванного потока инструкций со следующего такта без потерь промежуточных данных.

9 Язык ассемблера и компилятор LIM M2 (`limasm.py`)

Для разработки программного обеспечения, системных прошивок и операционных систем под архитектуру **LIM M2** реализован двухпроходный компилятор ассемблера `LIM2/sim/limasm.py`, транслирующий исходный код (`.asm` / `.lasm`) одновременно в бинарный образ памяти (`.bin`) и шестнадцатеричный дамп (`.hex`).

9.1 Именование регистров и суффиксы размера операндов

В языке ассемблера LIM M2 используется строгая, наглядная система обозначения регистров через префикс `$`:

- **Регистры общего назначения (16 бит):** обозначаются как `$0`, `$1`, `$2`, `$3`, `$4`, `$5`, `$6`, `$7` (для совместимости компилятор также принимает запись `R0`–`R7` и `$r0`–`$r7`).
- **Специальные системные регистры:**
  - `$ptreg` — 24-битный адресный регистр-указатель (PTREG);
  - `$freg` — 16-битный регистр состояния и флагов (FREG);
  - `$sreg` (или `$sp`) — 24-битный системный указатель стека (SREG);
  - `$vec0`–`$vec7` — регистры внутреннего стека векторов прерываний процессора.
- **Суффиксы размера данных (`.b` / `.w` / `.d`):** в инструкциях, где требуется явно указать размер обрабатываемых данных, после мнемоники через точку пишется суффикс:
  - `XXX.b` — **byte** (1 байт / 8 бит, аппаратный бит $\mathrm{SZ} = 0$);
  - `XXX.w` — **word** (1 слово / 16 бит, аппаратный бит $\mathrm{SZ} = 1$, используется по умолчанию);
  - `XXX.d` — **doubleword** (двойное слово / 24–32 бита, для полных 24-битных адресов и составных пар).

9.2 Арифметические и унарные регистровые инструкции

Вычислительные инструкции АЛУ работают исключительно с регистрами общего назначения, придерживаясь правила «сначала приёмник, затем источники»:

- **Трёхоперандная арифметика (`ADD`, `SUB`, `MUL`, `DIV`, `MOD`, 2 байта):**

```
ADD $0, $1, $2      ; $0 = $1 + $2 (также допустима запись: ADD R0, R1, R1)
SUB $3, $0, $1      ; $3 = $0 - $1
MUL $4, $1, $2      ; $4 = $1 * $2 (старшие 16 бит фиксируются в остатке АЛУ)
DIV $5, $4, $1      ; $5 = $4 / $1 (остаток от деления фиксируется в АЛУ)
MOD $6, $0, $2      ; $6 = $0 % $2 (прямое получение остатка от деления)
```

- **Унарные однобайтовые операции (`INC`, `DEC`, `NOT`, 1 байт):** кодируются ровно одним октетом (старшие 5 бит — опкод, младшие 3 бита — индекс регистра):

```
INC $0              ; $0 = $0 + 1 (1 байт, возможна запись INC R0)
DEC $1              ; $1 = $1 - 1 (1 байт, возможна запись DEC R1)
NOT $2              ; $2 = ~$2    (1 байт, побитовая инверсия)
```

9.3 Загрузка и выгрузка памяти (`LMR` и `LRM`): числа, ссылки `[...]` и пары регистров

В командах обмена с памятью `LMR` (Load Memory to Register) и `LRM` (Load Register to Memory) тип второго аргумента однозначно определяет аппаратный режим адресации (SRC / DST = 0..3):

1. **Просто число без скобок (непосредственное значение, SRC $= 0$ / DST $= 0$):** если аргумент записан просто числом, он трактуется как непосредственная константа в потоке кода сразу за инструкцией:

```
LMR.b $0, 0x42          ; Загрузить 1 байт (0x42) непосредственно в $0
LMR.w $1, 0x1234        ; Загрузить 16-битное слово (0x1234) непосредственно в $1
```

2. **Число или метка в квадратных скобках `[addr]` (прямая ссылка на данные в памяти, SRC $= 1$ / DST $= 1$):** квадратные скобки означают разыменование адреса (обращение к ячейке памяти по указанной ссылке):

```
LMR.w $0, [0x1000]      ; Прочитать 16-битное слово из памяти по адресу 0x1000 в
LRM.w $0, [0x1000]      ; Записать 16-битное слово из $0 в память по адресу 0x100
LMR.b $2, [0x8000]      ; Прочитать 1 байт по адресу 0x8000 в младший байт $2
```

3. **Ссылка через специальный регистр-указатель `[$ptreg]` (SRC $= 2$ / DST $= 2$, 2 байта):** если адрес данных находится в 24-битном регистре PTREG, он указывается в квадратных скобках. При установленном флаге `FREG.AIP` указатель автоматически инкрементируется на 1 (`.b`) или 2 (`.w`) байта после каждого обращения:

```
LMR.w $0, [$ptreg]      ; $0 = MEM16[$ptreg] (команда занимает всего 2 байта)
LRM.b $4, [$ptreg]      ; MEM8[$ptreg] = $4  (запись байта по указателю $ptreg)
```

4. **Составной 24-битный поинтер на основе пары регистров `[$hi:$lo]` (SRC $= 3$ / DST $= 3$, 2 байта):** позволяет сформировать полный 24-битный физический адрес из двух 16-битных регистров общего назначения (EA $= (((\$0 \ll 16) \mid \$1) \;\&\; 0$xFFFFFF). При чтении (LMR) старший регистр пары перезаписывается считанными данными:

```
LMR $0, [$0:$1]         ; EA = ($0 << 16) | $1, результат MEM[EA] загружается в $
                        ; (также поддерживается запись с запятой: LMR, $0, [$0:$1
LRM $0, [$0:$1]         ; Запись значения $0 по 24-битному адресу ($0 << 16) | $1
```

9.4 Манипуляция специальными регистрами (MSP) и многократное разыменование указателей

Инструкция MSP использует точечную нотацию субопераций и поддерживает как регистры, так и непосредственные константы и ссылки в квадратных скобках:

- **Модификация флагового регистра `$freg` битовыми масками (`MSP.and`, `MSP.or`, `MSP.xor`, `MSP.xnor`):**

```
MSP.and $freg, 0x0001   ; Наложить маску AND на флаговый регистр: FREG = FREG & 0x
MSP.or  $freg, 0x1000   ; Установить бит 12 (AIP — автоинкремент PTREG)
MSP.xor $freg, $2       ; Инвертировать биты FREG по маске из регистра $2
```

- **Прямая загрузка глухой настройки или поинтера (`MSP.ld`):**

```
MSP.ld $freg, 0x0000        ; Загрузить глухую настройку (0x0000) в регистр флагов
MSP.ld $ptreg, [$0:$1]      ; Собрать 32-битный поинтер в PTREG из пары ($0 << 16)
MSP.ld $sreg, [$0:$1]       ; Загрузить 32-битный указатель стека SREG из пары ($0
MSP.ld $ptreg, 0xA000       ; Загрузить непосредственный адрес 0xA000 в PTREG
MSP.ld $ptreg, [0xA000]     ; Загрузить в PTREG поинтер, лежащий в памяти по ссылк
MSP.ld $sreg, 0x3F00        ; Инициализировать указатель стека SREG
MSP.add $ptreg, 0x0010      ; Сместить указатель PTREG на +16 байт (аналогично MSP
```

- **Нативное многократное разыменование цепочки указателей через `$ptreg`:** В архитектуре LIM M2 нативное аппаратное разыменование указателя «сам в себя» (PTREG $\leftarrow$ MEM[PTREG]) работает напрямую с регистром PTREG за одну 2-байтовую инструкцию `MSP.ld $ptreg, [$ptreg]` (в то время как для обычных регистров пришлось бы вручную выгружать и перезагружать адрес). Поэтому при обходе многоуровневых указателей (списков, деревьев, таблиц дескрипторов) базовый адрес загружается в `$ptreg`, после чего команда `MSP.ld $ptreg, [$ptreg]` выполняется столько раз (последовательно или в цикле), сколько уровней разыменования требуется пройти:

```
MSP.ld $ptreg, 0xA000       ; Загружаем начальный адрес цепочки указателей (0xA000
MSP.ld $ptreg, [$ptreg]     ; 1-е разыменование: PTREG = *0xA000
```

```
MSP.ld $ptreg, [$ptreg]     ; 2-е разыменование: PTREG = **0xA000
MSP.ld $ptreg, [$ptreg]     ; 3-е разыменование: PTREG = ***0xA000
LMR.w  $0, [$ptreg]         ; Читаем финальные данные по разыменованному адресу в
```

Или в виде цикла, где счётчик в регистре $1 задаёт глубину разыменования:

```
    MSP.ld $ptreg, 0xA000   ; Начальный адрес цепочки
    LMR.w  $1, 3            ; Количество разыменований = 3
deref_loop:
    MSP.ld $ptreg, [$ptreg] ; PTREG <- MEM16[PTREG]
    DEC    $1               ; $1 = $1 - 1
    JNZ    deref_loop       ; Повторять, пока $1 != 0
    LMR.w  $0, [$ptreg]     ; Чтение целевого значения в $0
```

9.5 Использование CLI-компилятора (.bin и .hex)

Компилятор запускается из командной строки и автоматически создаёт как бинарный файл (.bin), так и шестнадцатеричный файл (.hex), готовые к запуску в симуляторе sim.py:

```
# Компиляция программы с выводом таблицы меток и дизассемблированного листинга (-l)
python3 LIM2/sim/limasm.py LIM2/sim/demo.asm -l

# Явное указание имен выходных .bin и .hex файлов и базового адреса:
python3 LIM2/sim/limasm.py program.asm -o build/program.bin -x build/program.hex -b

# Прямой запуск .asm, .bin или .hex файла в интерактивном симуляторе LIM M2:
python3 LIM2/sim/sim.py LIM2/sim/demo.asm -i
```

10 Заключение

Архитектура **LIM M2** сочетает в себе проверенную надёжность многотактового микропрограммного управления и современную эффективность мультиплексорного тракта без третьего состояния. Такое решение обеспечивает чистейшие цифровые фронты, исключает паразитные токи и гарантирует непревзойдённую стабильность при масштабировании тактовых частот от 200 МГц до пределов физического кремния.